

Unit-III

Introduction to UML: Importance of Modeling, Principles of Modeling, Conceptual model of the UML, Architecture, Diagrams: Class Diagrams, Forward and Reverse engineering for Class Diagram. Use case Diagrams, Sequence diagrams.

What Is the UML?

- The UML is a graphical language for
 - Visualizing
 - Specifying
 - Constructing
 - Documentingthe work products of software systems.



- The Unified Modelling Language (UML) is an industry standard language.
- UML is a process-independent language.

Why We Model ?

- We build models so that we can better understand the system we are developing
- Analyse the problem-domain
 - simplify reality
 - capture requirements
 - visualize the system in its entirety
 - specify the structure and/or behaviour of the system
- Design the solution
 - document the solution - in terms of its structure, behaviour, etc.

The Importance of Modeling

- A model is “a complete description of a system”



Model

- A model is a simplification of reality.
- A model provides the blueprints of a system.
- A model may be structural(emphasizing the organization of the system), or it may be behavioural, (emphasizing the dynamics of the system).

Why Model?

- Modeling achieves four aims:
 - Helps you to **visualize a system**.
 - Permits you to **specify the structure or behavior of a system**.
 - Gives you a **template that guides you in constructing a system**.
 - **Documents the decisions** you have made.
- You build models of complex systems because you cannot comprehend such a system in its entirety.

The UML Is a Language for Visualizing

- Communicating conceptual models to others is prone to error unless everyone involved speaks the same language.
- There are things about a software system you can't understand unless you build models.
- An explicit model facilitates communication.

The UML Is a Language for Specifying

- The UML builds models that are precise, unambiguous, and complete.

The UML Is a Language for Constructing

- UML models can be directly connected to a variety of programming languages.
 - Maps to Java, C++, Visual Basic, and so on
 - Permits **forward engineering**: Generation of Source Code from UML Model.
 - Permits **reverse engineering**: Generation of UML Model from Source Code

The UML Is a Language for Documenting

- The UML addresses documentation of system architecture, requirements, tests, project planning, and release management.

UML Application Areas

- The UML is intended primarily for Software intensive systems. It has been used effectively for such domains as follows:
- Enterprise information systems
- Banking and financial services
- Telecommunications
- Transportation
- Defense
- Medical , Electronics, scientific and Distributed web-based services.

Principles of Modeling

- The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped.
- Every model may be expressed at different levels of precision.
- The best models are connected to reality.
- No single model is sufficient. Every nontrivial system is best approached through a small set of nearly independent models.

Conceptual Model of UML

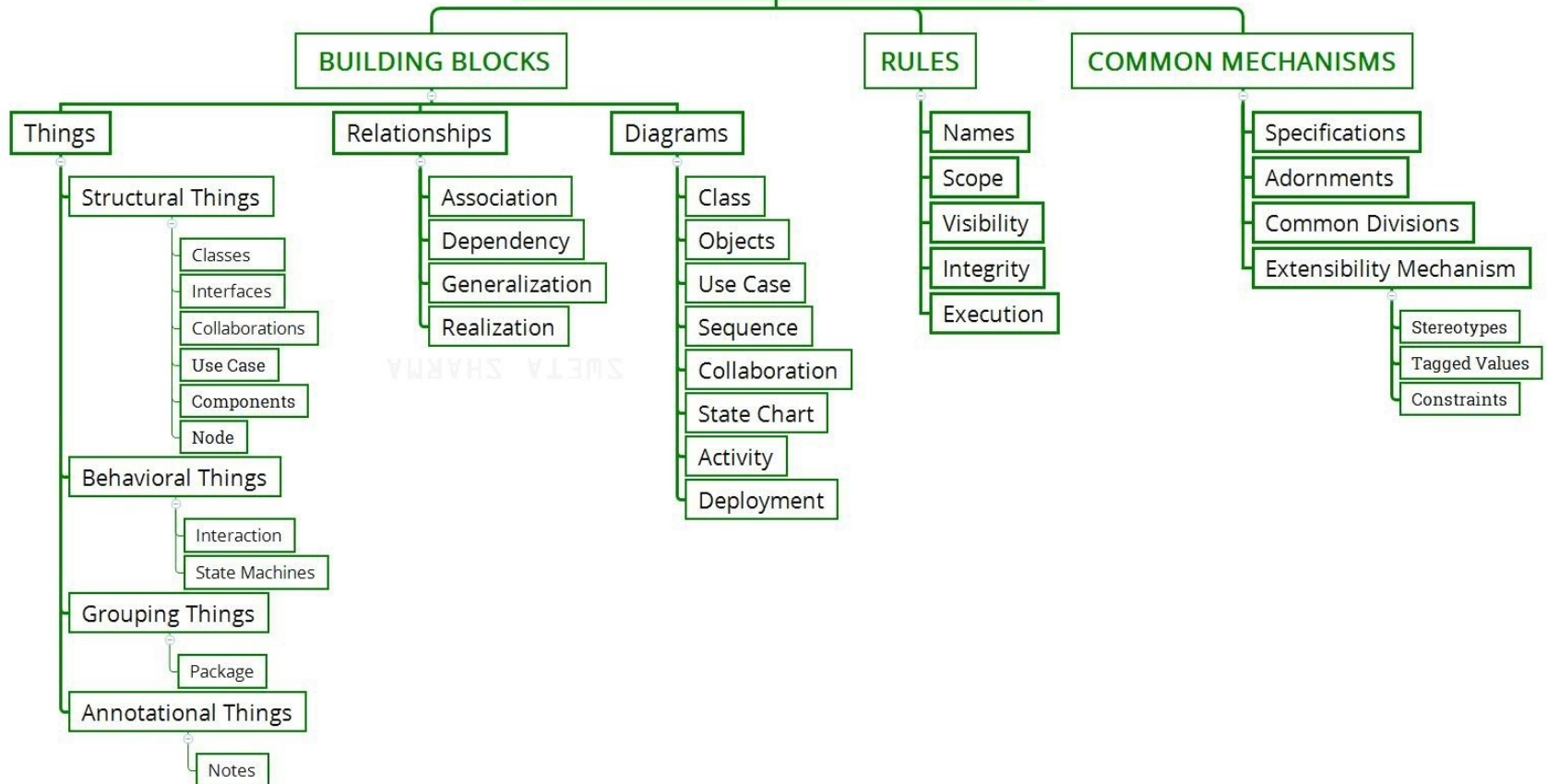
A Conceptual Model of the UML

- To understand the UML, a conceptual model of the language is needed to form and it requires the **three major elements**:
 1. **Building blocks** of the UML.
 2. **Rules** that dictate how these building blocks may be put together.
 3. **Common mechanisms** that apply throughout the UML.

1. Building blocks of the UML

- The UML consists of three kinds of building blocks:
 - a) Things*- These are the **abstractions** that are first-class citizens in a model.
 - b) Relationships*-Tie the above things together.
 - c) Diagrams*-Group interesting collections of things,

CONCEPTUAL MODEL OF UML



Things in the UML

- There are 4 kinds of things in the UML
 1. Structural Things
 2. Behavioral Things
 3. Grouping Things
 4. Annotational Things

1. Structural Things

- Structural things are the **nouns of UML models**. These are mostly **static** parts of a model, representing elements that are either conceptual or physical.

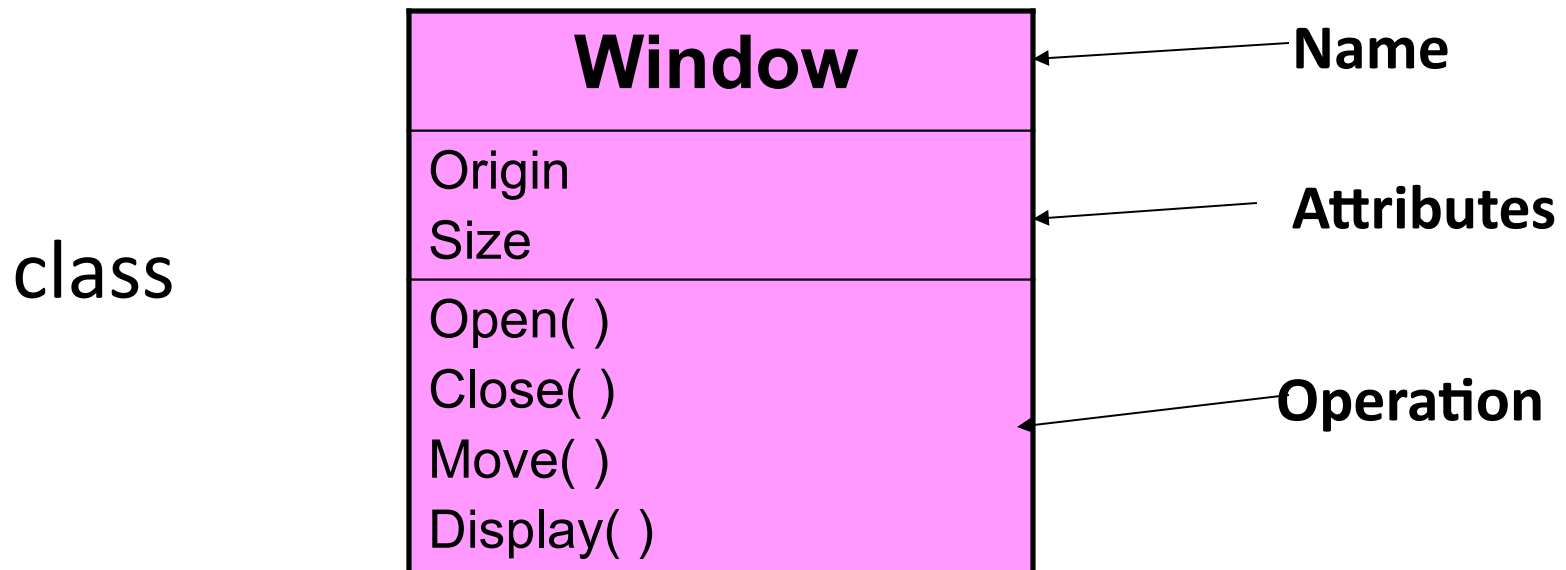
There are 7 kinds of Structural things:

1. Class
2. Interface
3. Collaboration
4. Use case
5. Active class
6. Component
7. Node

(1) Class:-

A class is a **description of a set of objects** that share the same attributes, operations, relationships and semantics.

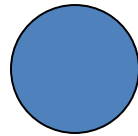
A class implements one or more interfaces.



(2) Interface:-

- Interface is a collection of operations that specify a service of a class or component.
- An interface defines a set of operation specifications but never a set of operation implementations.
- Graphically, an interface is represented as a circle with its name.

Interface Notation



ISpelling

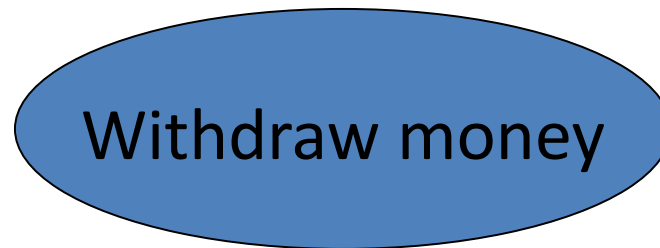
(3) Collaboration:-

- A collaboration defines an interaction and is a society of roles and other elements that work together to provide some cooperative behavior.
- Graphically a collaboration is represented as an ellipse with dashed lines, usually including only its name.

(4) Use case:-

- A Use case is a **description of set of sequence of actions** that a system performs, which results a value to a particular actor.

- A use case is realized by a collaboration.



Use case

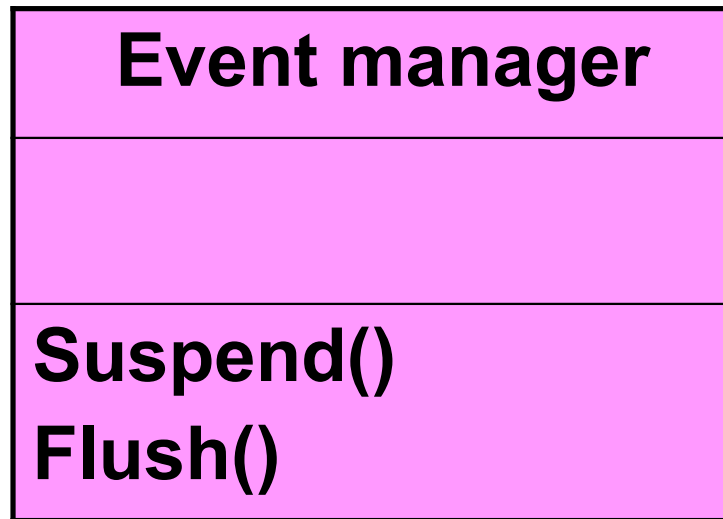
- The remaining three things- **Active classes, components and nodes**- all are class like, ie., they also describe a set of objects that share the same attributes, operations, relationships and semantics.

(5) Active class:-

- An active class is a class whose objects own one or more processes or threads and therefore can initiate control activity.
- **Active Class** initiate and control the flow of activity, while passive classes store data and serve other classes.

We illustrate active classes with a thicker border.

- Graphically, an active class is represented like a class, but with **heavy lines**, usually including its name, attributes and operations.

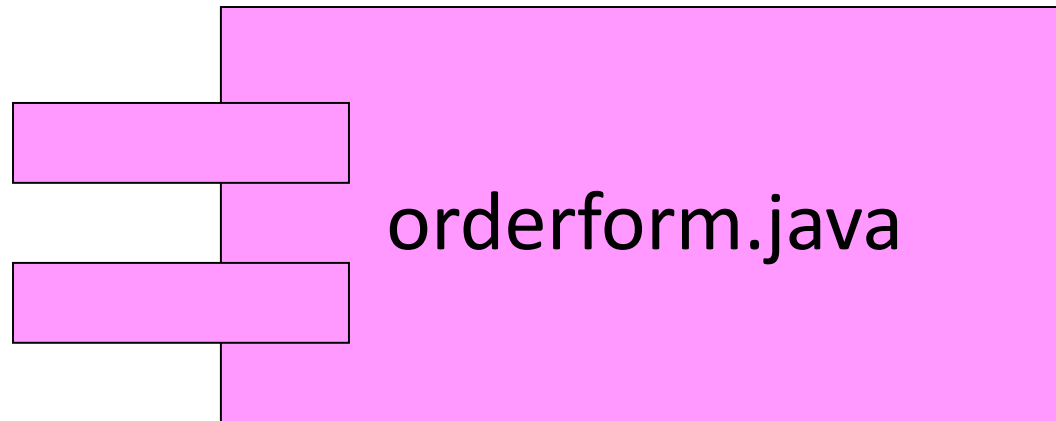


Active class

- The remaining two elements –component and nodes- they represent physical things, where as previous five things (class, interface, collaboration, usecase, active class) represent conceptual or logical things.

(6) Component:-

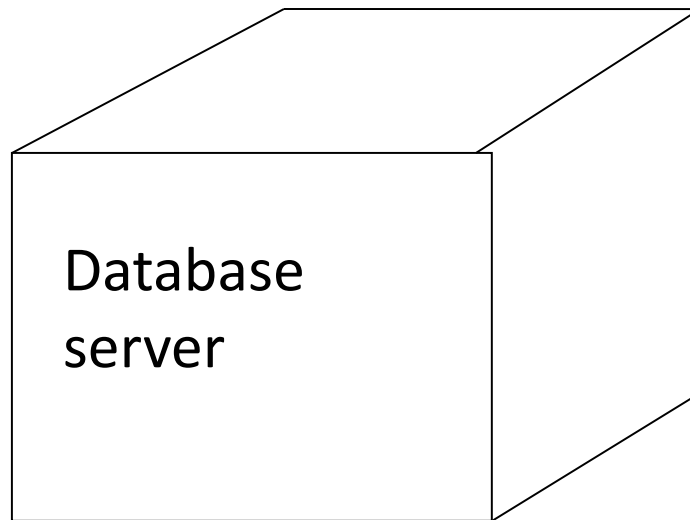
- A component is a **physical and replaceable part** of a system that conforms to and provides the **realization of a set of interfaces**.
- A **component represents** the **physical packaging of logical elements** such as classes, interfaces and collaborations.



Component

(7) Node:-

- A node is **physical element** that **exists at run-time** and represents a computational resource, generally **having some memory and processing capability**.
- A set of components may reside on a node and may also migrate from node to node.



Database
server

Node

2. Behavioral Things

- These are the **dynamic parts of UML models**.
- These are the **verbs of a model** representing the behavior over **time and space**.
- There are two types of behavioral things
 1. Interaction
 2. State machine.

(1) Interaction:-

- An Interaction is a **behavior** that comprises a **set of messages** exchanged **among a set of objects** within a particular context **to perform a specific purpose.**



- An interaction involves a number of other elements, including messages, action sequences and links.
(connection between objects.)

2. State machine:-

A State machine is a behavior that specifies the **sequences of states** an object or an interaction goes through during its lifetime in response to events, together with its responses to those events.



waiting

State

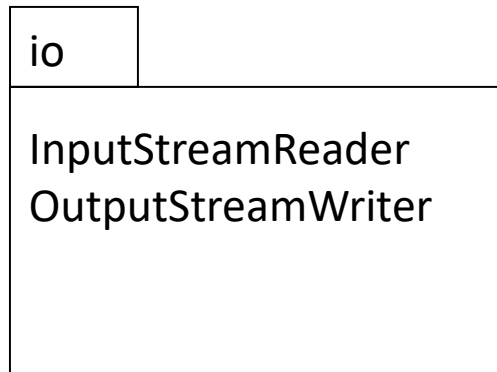
- A State machine involves a number of other elements, including **states**, **transitions**(the flow from state to state), **events** (things that trigger a transition), and **activities** (the response to a transition).

3. Grouping Things

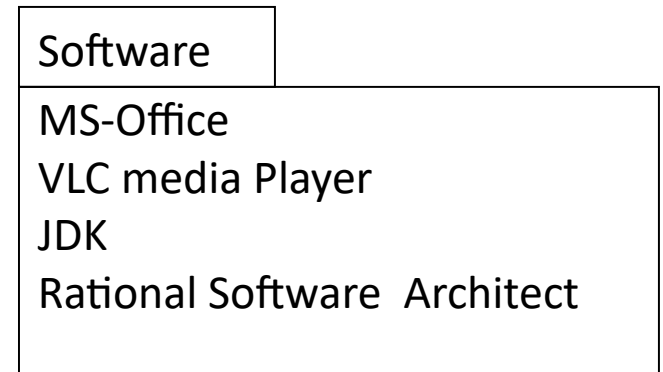
- Grouping things are the organizational parts of UML models.
- In all, there is one primary kind of grouping thing, namely, Package.

- Package:- A Package is a general-purpose mechanism for organizing elements into groups.
- Structural things, behavioral things and even other grouping things that may be placed in a package.

- Ex.1



- Ex. 2

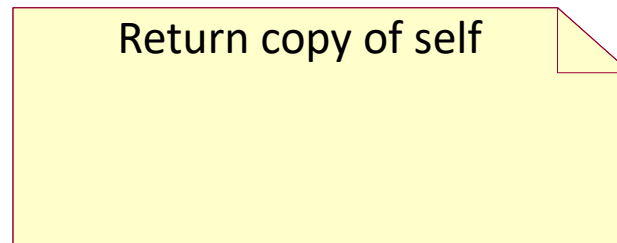


Package

4. Annotational Things

- Annotational Things are the explanatory parts of the UML models.
- These are the comments you may apply to describe and remark about any element in a model.
- There is one primary kind of annotational thing, called a Note.

- Note:- A Note is simply a symbol for representing constraints and comments attached to an element.
- A note is rendered as a rectangle with a dog-eared corner.



Note

Relationships

- A relationship is a connection among UML things.
- In object-oriented modeling, the three most important relationships are **dependencies, generalization and association.**

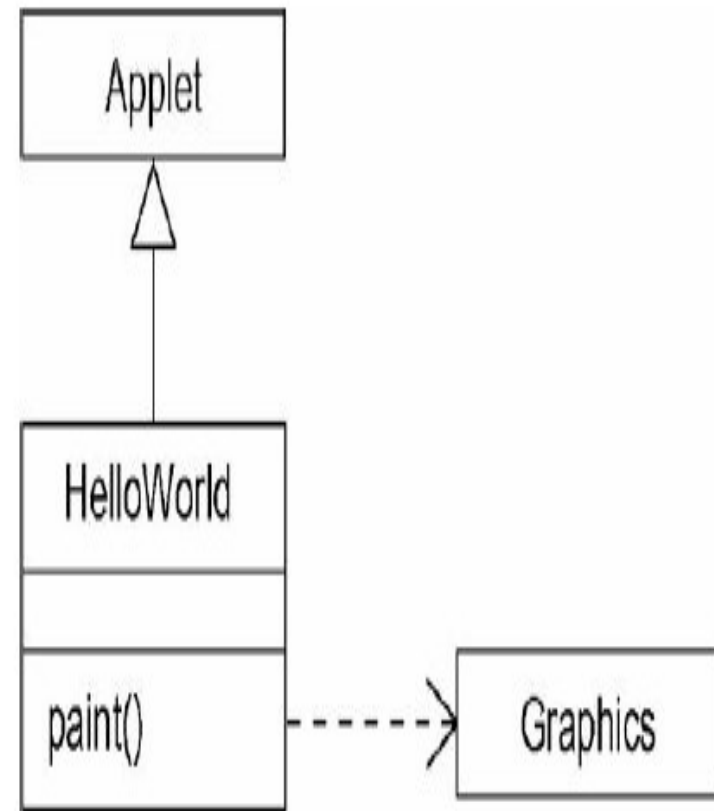
Realization relation is used to specify the relationship between **interface and class.**

Relationships in models and Java Code

-
- In Java, the applet for printing "Hello, World!" in a Web browser is quite simple:

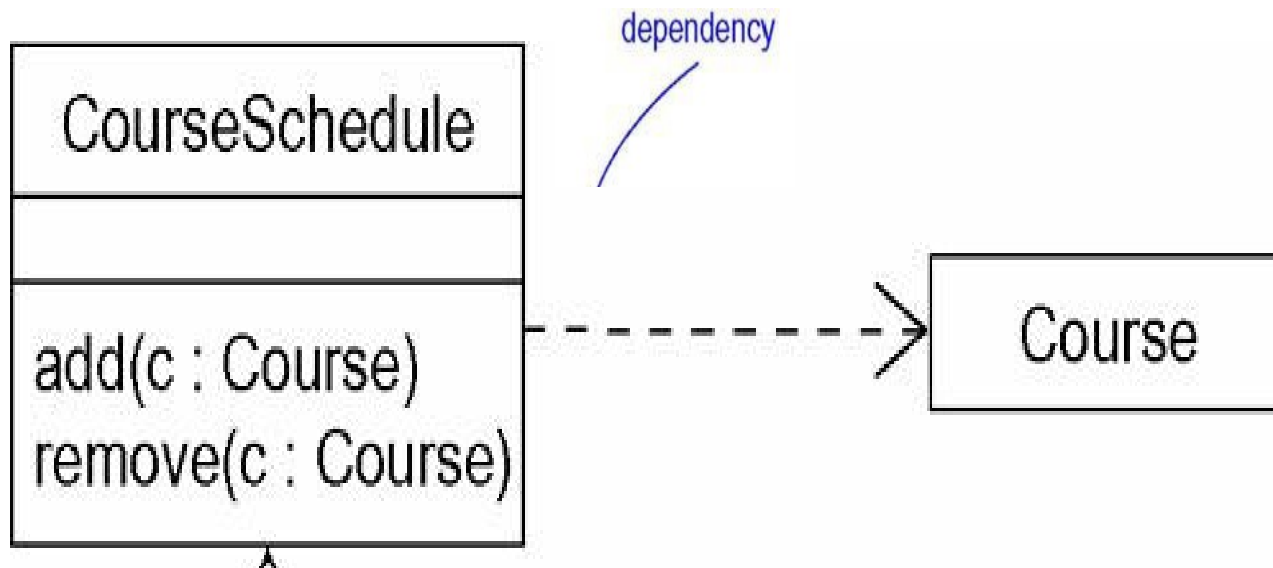
```
import java.awt.Graphics;
```

```
class HelloWorld extends java.applet.Applet  
{  
    public void paint (Graphics g)  
    {  
        g.drawString("Hello, World!", 10, 10);  
    }  
}
```



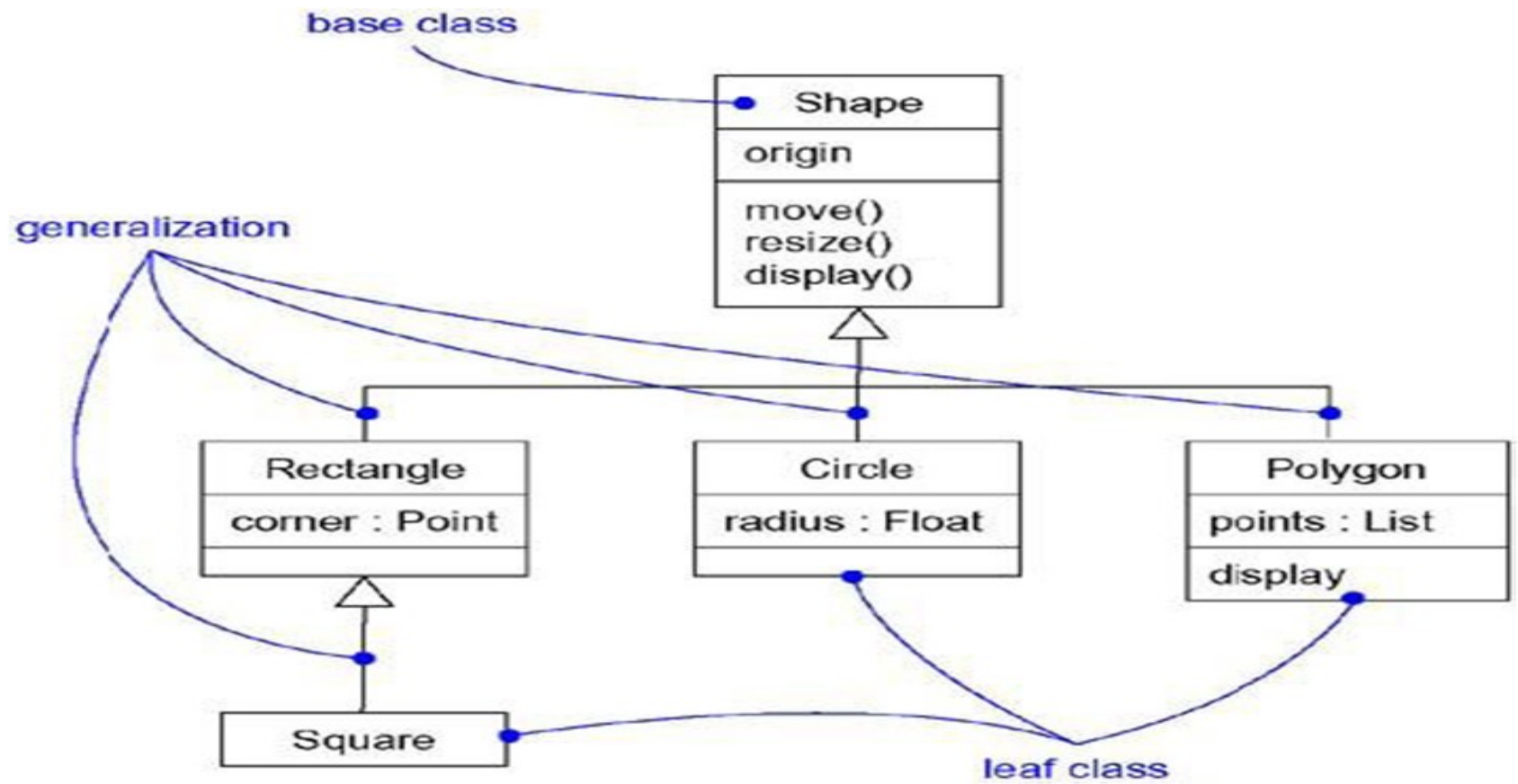
(1) DEPENDENCY

- A dependency is a **using relationship** that states that a change in specification of one thing may affect another thing that uses it.
- Graphically, a dependency is rendered as a dashed directed line, **directed to the thing being depended on.**



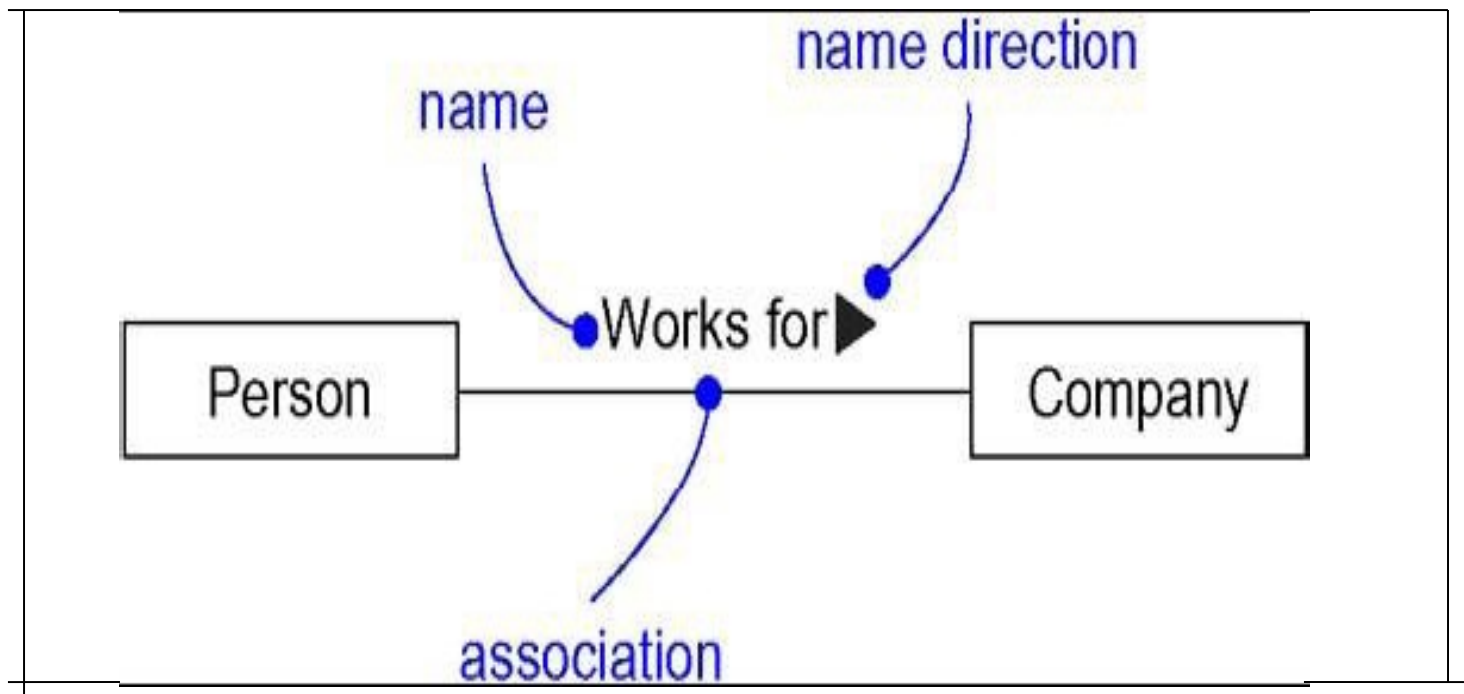
(2) GENERALIZATION

- A Generalization is relationship between a **general thing**(called the Super class or parent) and a more **specific kind of that thing**(called the Subclass or child).
- Generalization is sometimes called **“is-a-kind of”** relationship.
- The method of a child has the same signature as method in a parent, then the child method overrides the method in a parent; this is known as **Method Overriding**.
- Use generalizations when you want to show **parent/child relationship**.

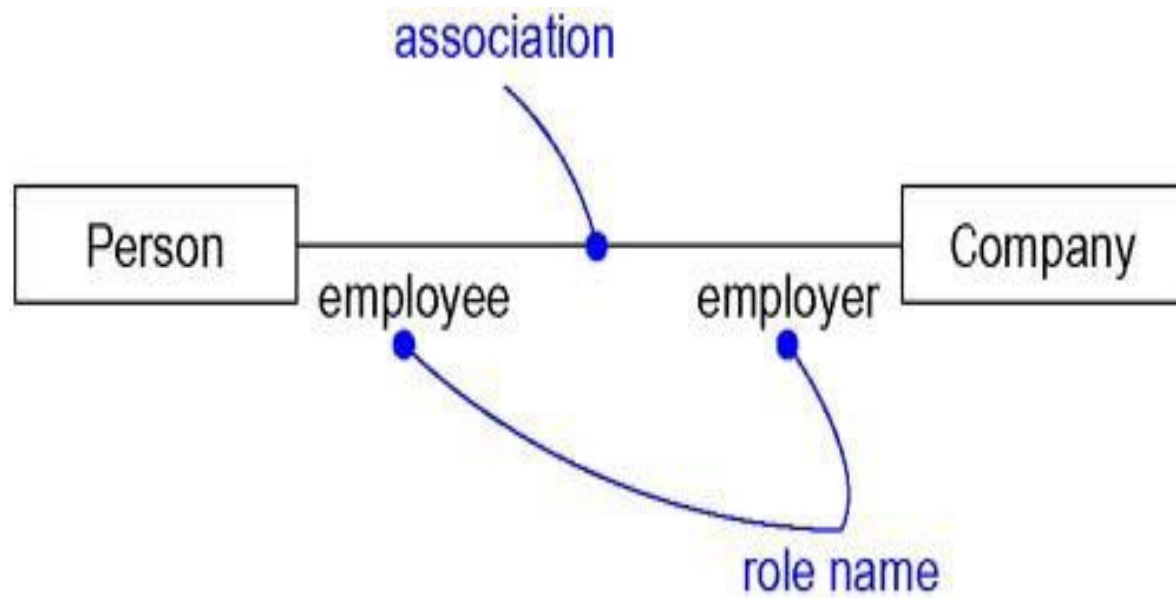


(3) ASSOCIATION

- An Association is a structural relationship that specifies that **objects of one thing are connected to objects of another.**
- Given an association **connecting two classes**, you can navigate from an object of one class to an object of the other class and vice versa.

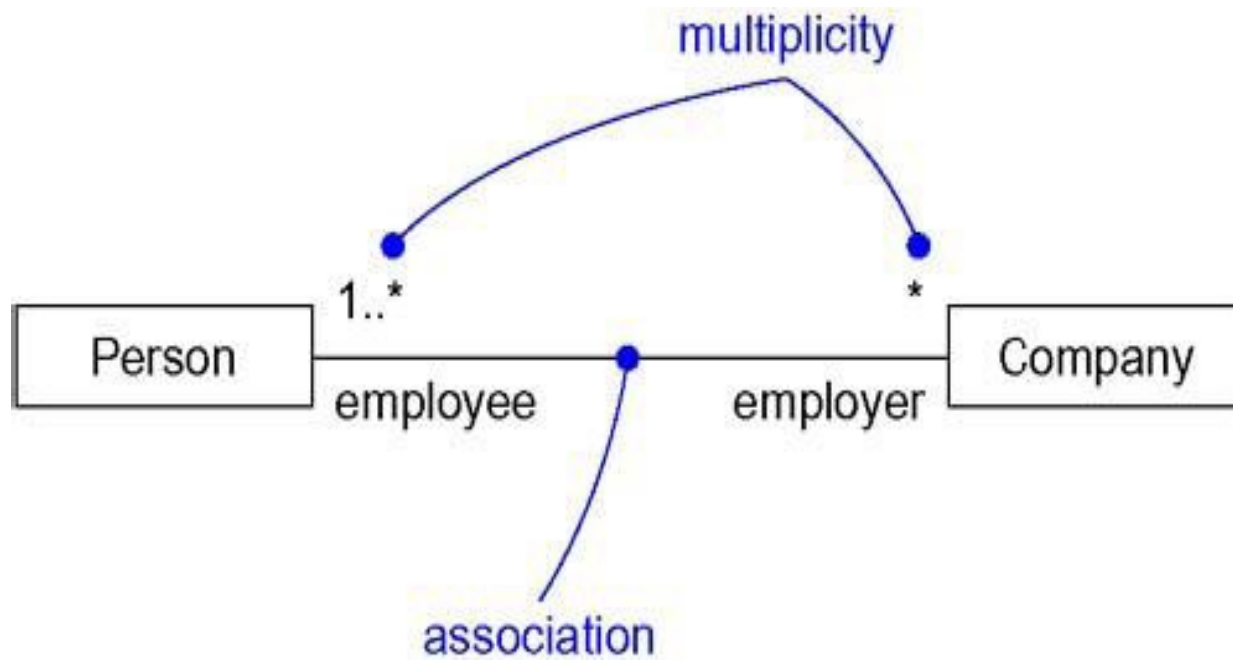


NAME



ROLE

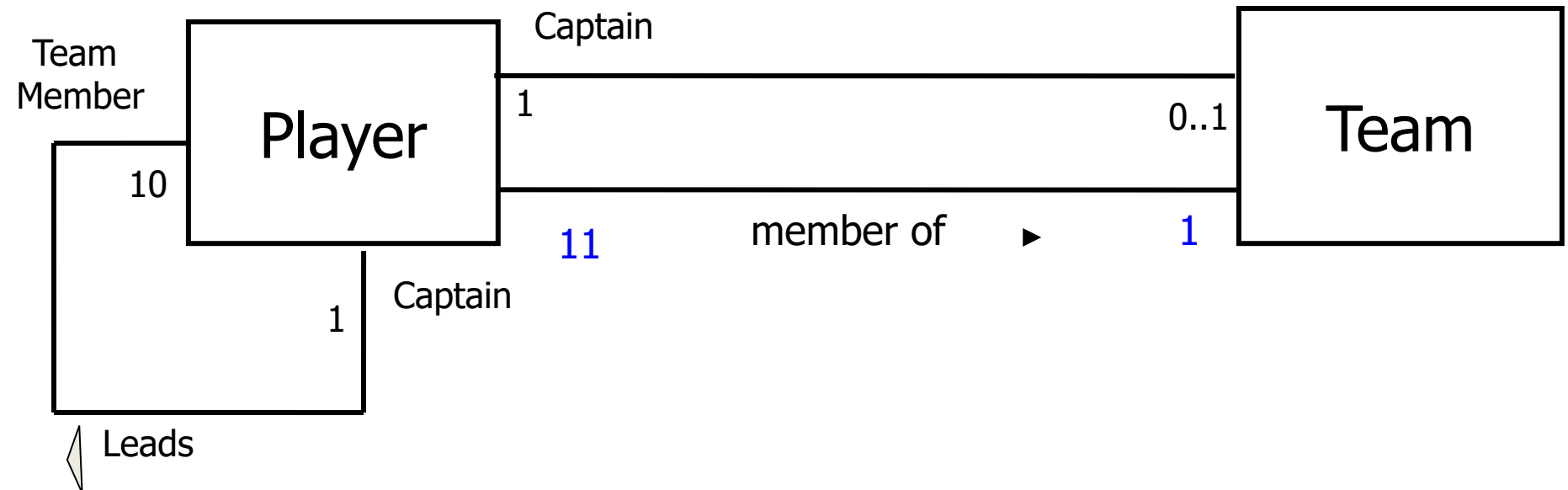
- **MULTIPLICITY:-**
- An association represents a **structural relationship among objects.**
- To state how many objects may be connected across an instance of an association.
- This “**how many**” is called the **multiplicity** of an association’s role.



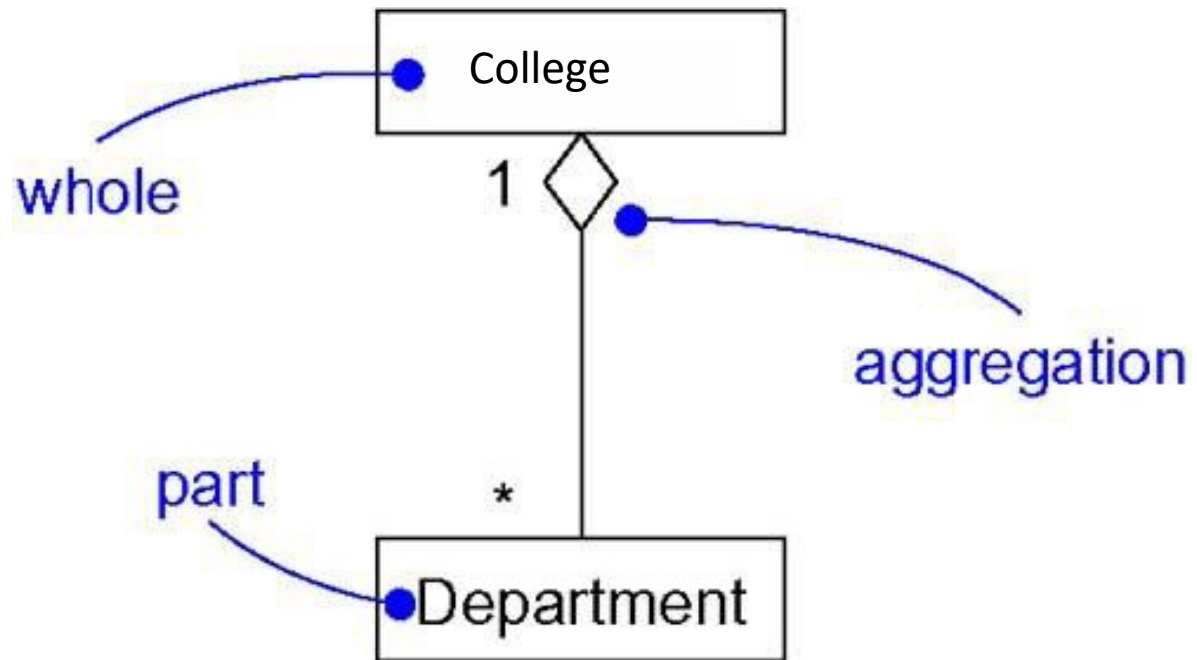
MULTIPLICITY

Association - Multiplicity

- A cricket team has **11** players. One of them is the captain.
- A player can play only for **one** Team.
- The **captain** leads the **team members**.



- **AGGREGATION:-**
- A plain association between two classes represents a structural relationship between peers.
- To model a “**Whole/part**” relationship, in which one class represents a large thing(the “whole”), which consists of smaller things (the “Parts”). This kind of relationship is called **AGGREGATION**, which represents a “**has-a**” relationship, meaning that an object of the whole has objects of the part.

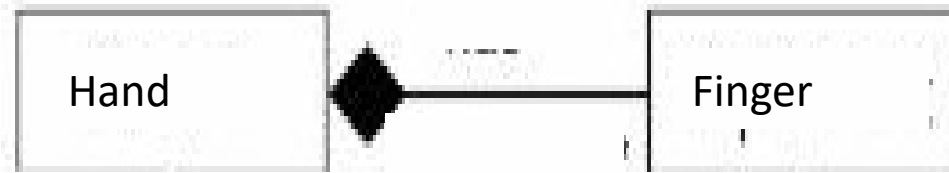


Aggregation

More Examples

- For instance, we can model an aggregation between classes **DegreeProgram** and **Course**, since a **course** is part of a **degree program** and a **course** can be shared among two or more **degree programs** (e.g. an engineering degree could share a C programming course with a computer science degree).
- *Use This Relationship Between*
- Body.....Blood
- Pen.....tip, barrier, ink reservoir etc.

- Aggregation is really just a special kind of association and is specified by adorning a plain association with an open diamond at the whole end.
- Composition:-
- Composition is a **special form of aggregation** within which the parts are **inseparable from the whole**.



Ex:

- Blood....BloodCells.
- Body....Heart
- Body....Brain

Realization

- A *realization* is a semantic relationship between classifiers in which one classifier specifies a contract that another classifier guarantees to carry out.
- Graphically, a realization is rendered as a dashed directed line with a large open arrowhead pointing to the classifier that specifies the contract.

- Realization is sufficiently different from dependency, generalization, and association relationships that it is treated as a separate kind of relationship.
- You'll use realization in the context of interfaces

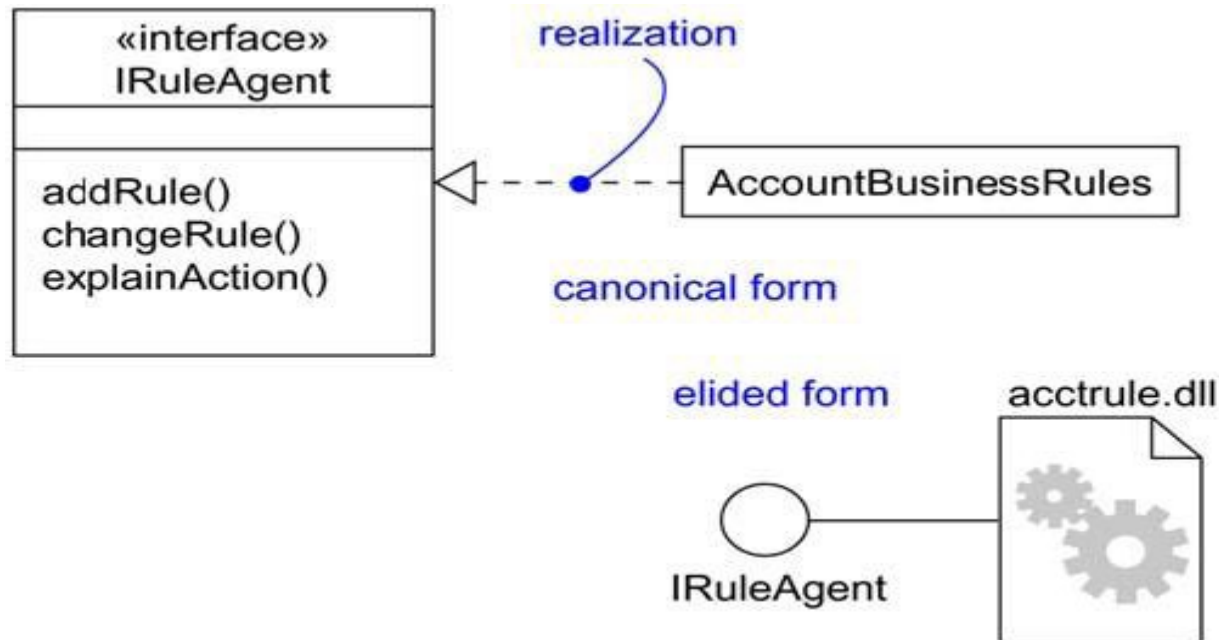
- Realization is used to specify the relationship between an interface and the class or component that provides an operation or service.
- An interface is a collection of operations that are used to specify a service of a class or a component.

Therefore, an interface specifies a contract that a class or component must carry out.

An interface may be realized by many such classes or components, and a class or component may realize many interfaces.

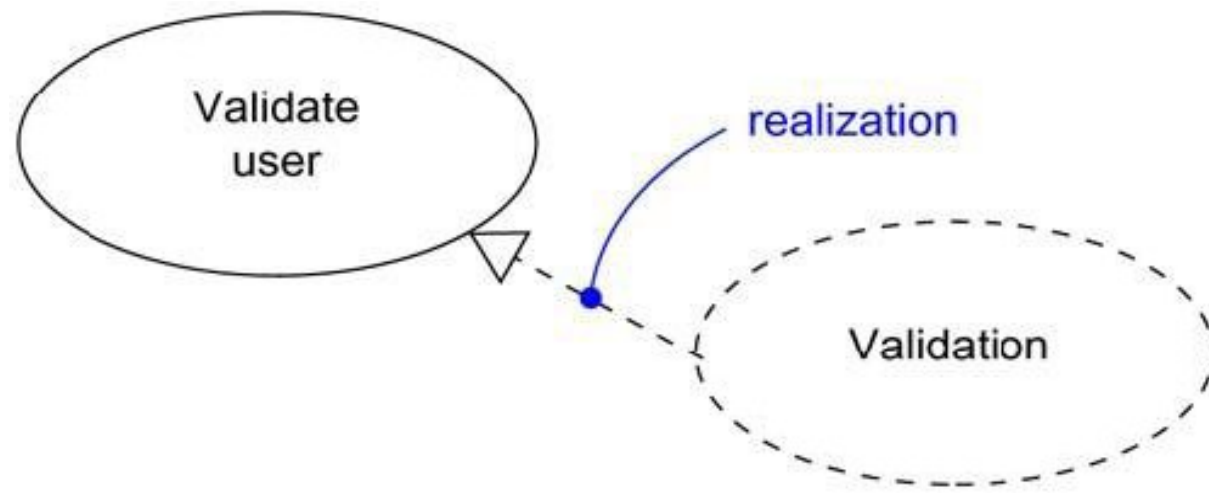
- Note that you can represent realization in two ways: in **the canonical form** (using the interface stereotype and the dashed directed line with a large open arrowhead) and in **an elided form** (using the interface lollipop notation).

Realization of an Interface



- You'll also use realization to specify the relationship between a **use case and the collaboration** that realizes that use case,
- In this circumstance, you'll almost always use the canonical form of realization.

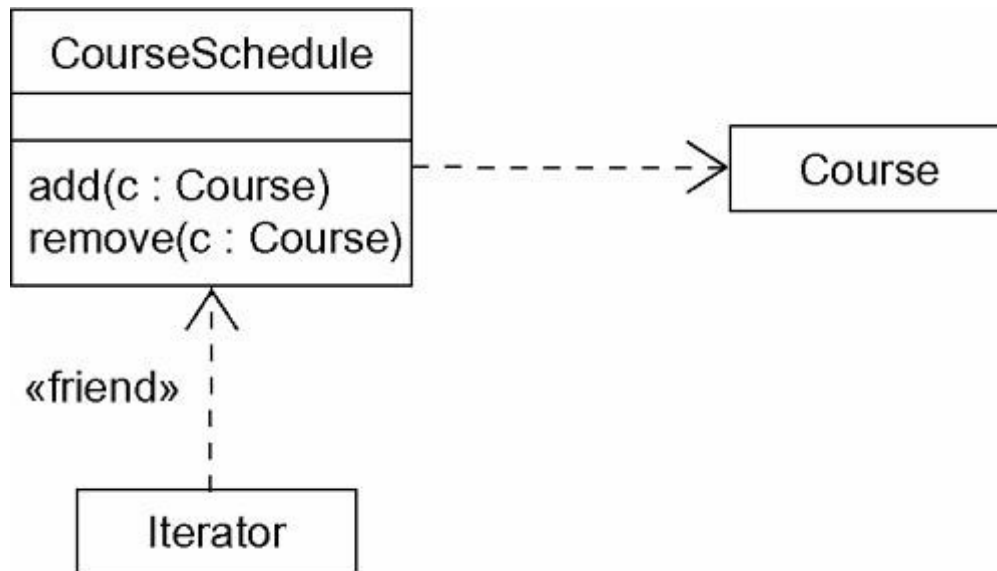
Realization of a Use Case



COMMON MODELING TECHNIQUES

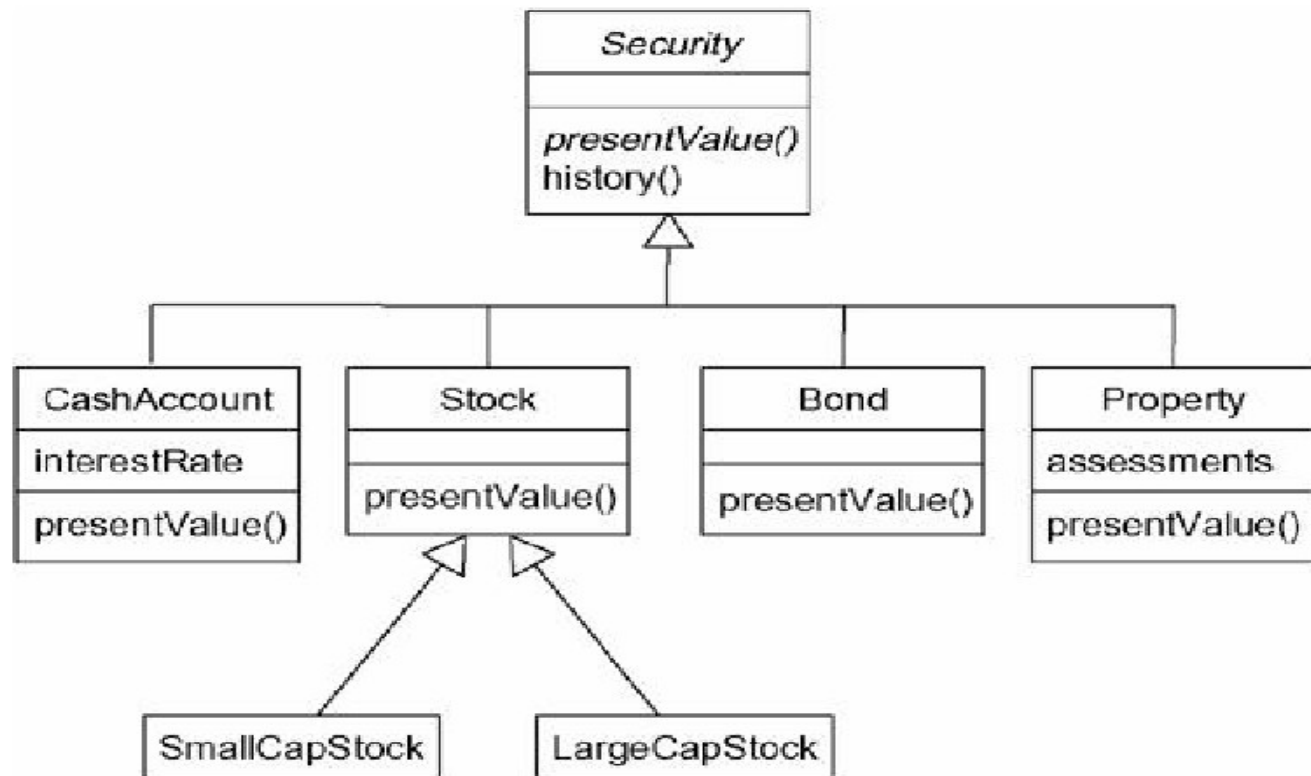
Modeling Simple Dependencies:-

- Create a Dependency pointing from the class with the operation to the class used as a parameter in the operation.



Modeling Single Inheritance:-

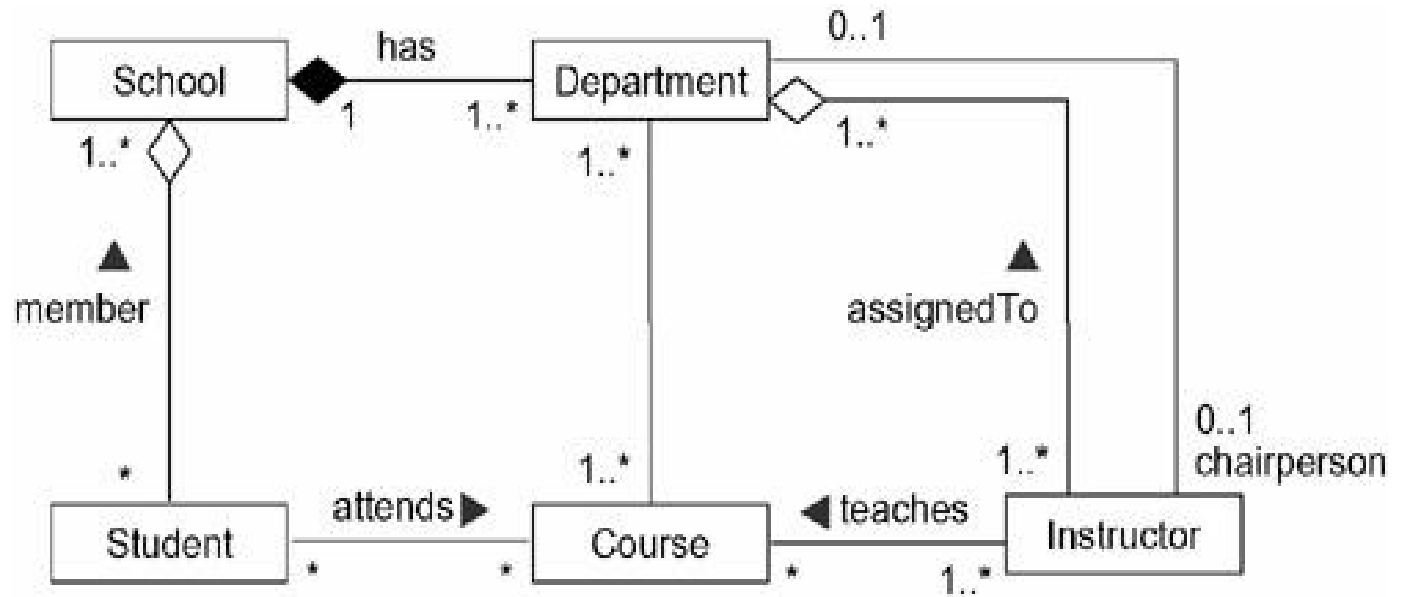
- Given a set of classes, look for responsibilities, attributes and operations which are common to two or more classes.
- Elevate these common responsibilities, attributes and operations to a more general class. If necessary create a new class to which we can assign these elements.
- Specify that the more-specific classes inherit from more-general class by placing a generalization relationship that is drawn from each specialized class to its more-general parent.



Inheritance Relationships

Modeling Structural Relationships:-

- For each pair of classes, if we need to navigate from objects of one class to objects of another class, specify an association between the two.
- For each of these associations, specify a multiplicity as well as role names.
- If one of the classes in an association is structurally or organizationally a whole compared with the classes at the other end look like parts, mark this as an aggregation by adorning the association at the end near the whole.



Structural Relationships

Diagrams in the UML

- A *diagram* is the graphical presentation of a set of things and relationships.
- We draw diagrams to visualize a system from different perspectives.
- Diagrams in the UML are broadly, classified into 2 types, namely
 - 1. Structural Diagrams and
 - 2. Behavioral Diagrams

Structural Diagrams (4)

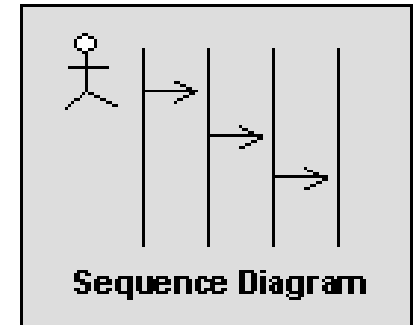
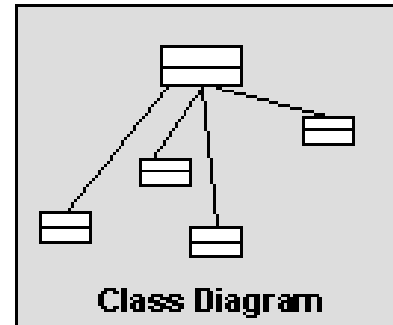
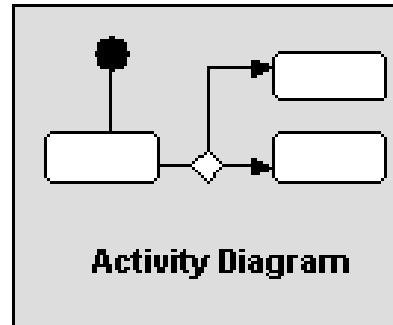
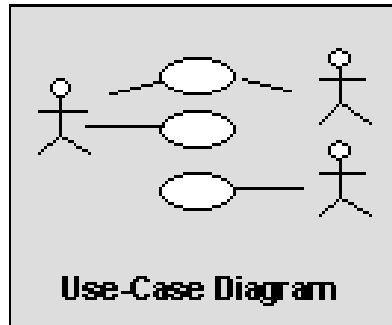
1. Class Diagram
2. Object Diagram
3. Component Diagram
4. Deployment Diagram

Behavioral Diagrams(5)

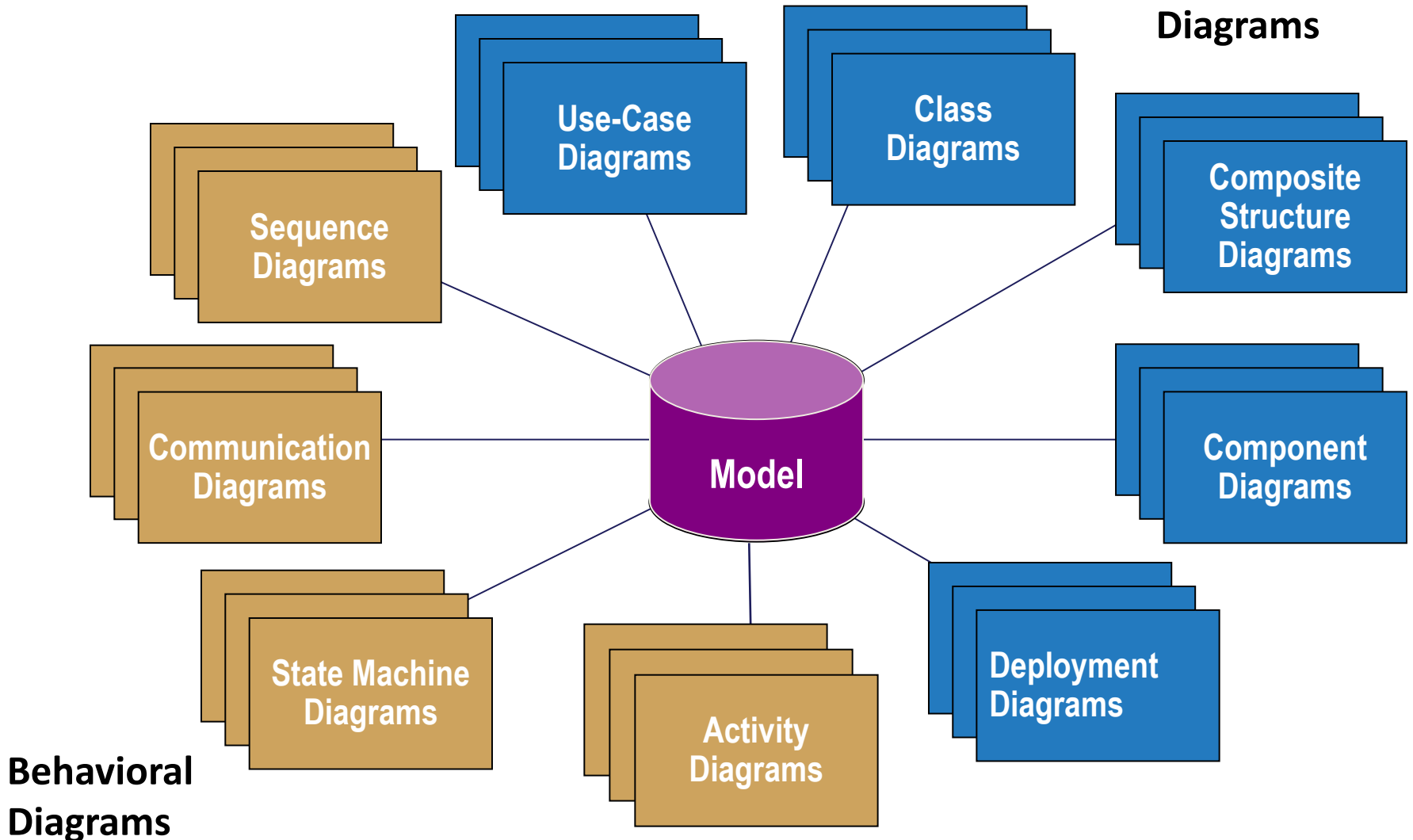
1. Usecase Diagram
2. Sequence Diagram
3. Collaboration Diagram
4. Activity Diagram
5. Statechart Diagram

Diagrams

- Diagrams graphically depict a view of a part of your model.
- Different diagrams represent different views of the system that you are developing.
- A model element will appear on one or more diagrams.



UML Diagrams in Software Architecture



Structural Diagrams

- The UML's four structural diagrams represent the **static aspects** of a system.
- The static aspects of a system means the existence and placement of things.

- The UML's structural diagrams are roughly organized around the major groups of things like
 1. **Class diagram**---Classes, interfaces, and collaborations
 2. **Object diagram**--- Objects
 3. **Component diagram**---Components
 4. **Deployment diagram**---Nodes

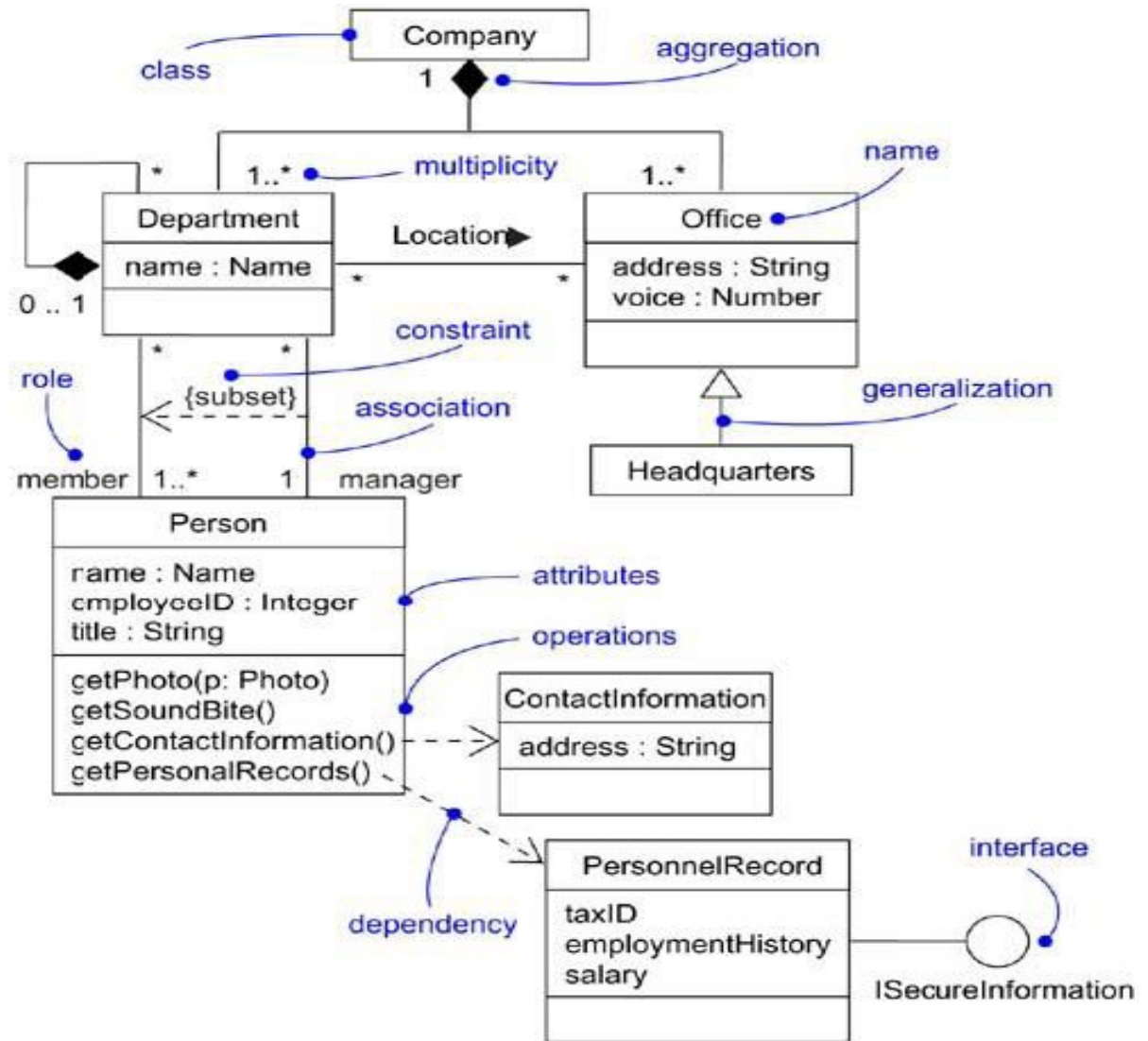
Behavioral Diagrams

- The UML's five behavioral diagrams represent the dynamic aspects of a system.
- Dynamic aspects of a system means **changing parts of the system.**
- The dynamic aspects of a software system include the flow of messages between the objects over time and space.

1. **Use case diagram**--- Organizes the behaviors of the system.
2. **Sequence diagram** ---Focused on the time ordering of messages.
3. **Collaboration diagram**---Focused on the structural organization of objects that send and receive messages.
4. **Statechart diagram**---Focused on the changing state of a system driven by events.
5. **Activity diagram**---Focused on the flow of control from activity to activity.

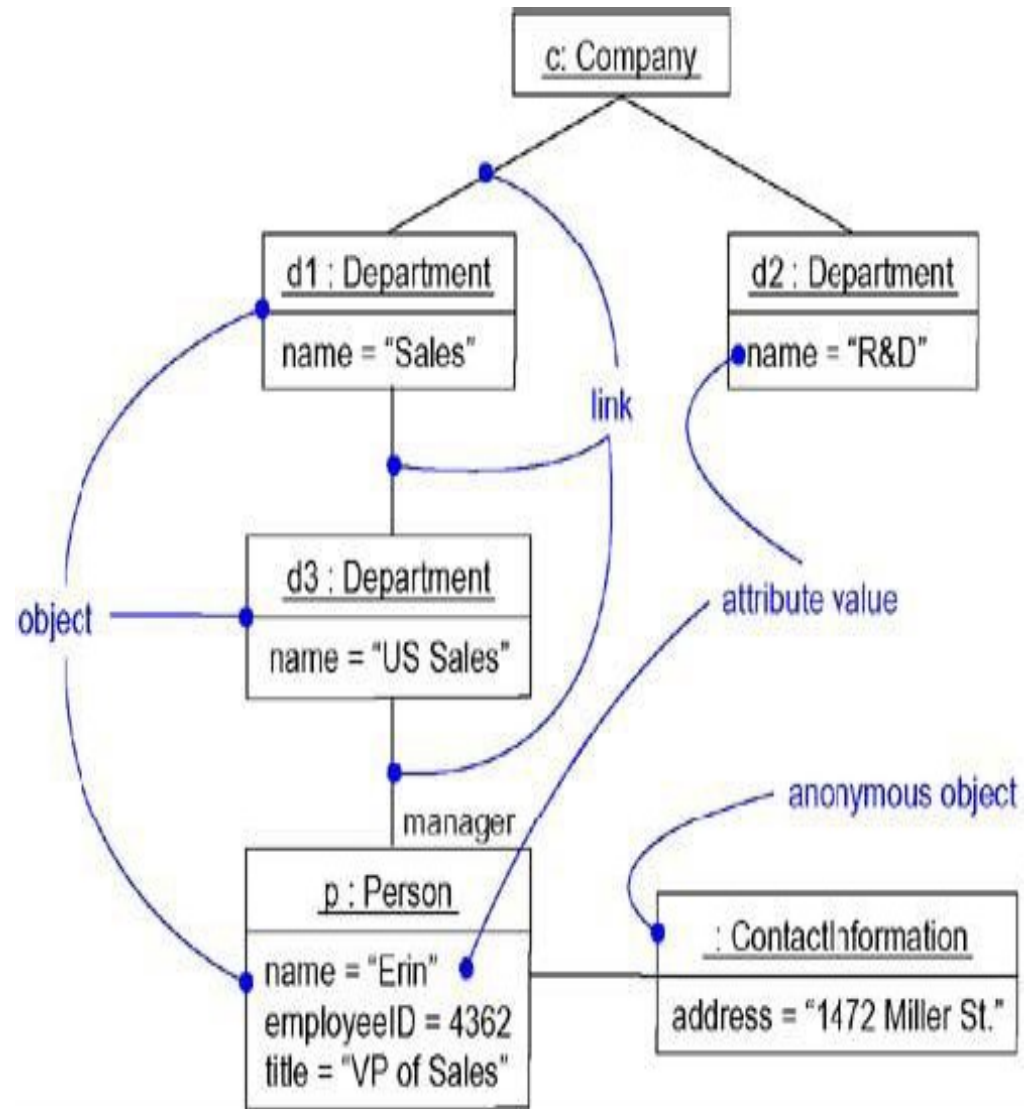
1. Class Diagram

- A **class diagram** shows a set of **classes, interfaces, and collaborations** and **their relationships**.
- Class diagrams are used to illustrate the **static design view** of a system.



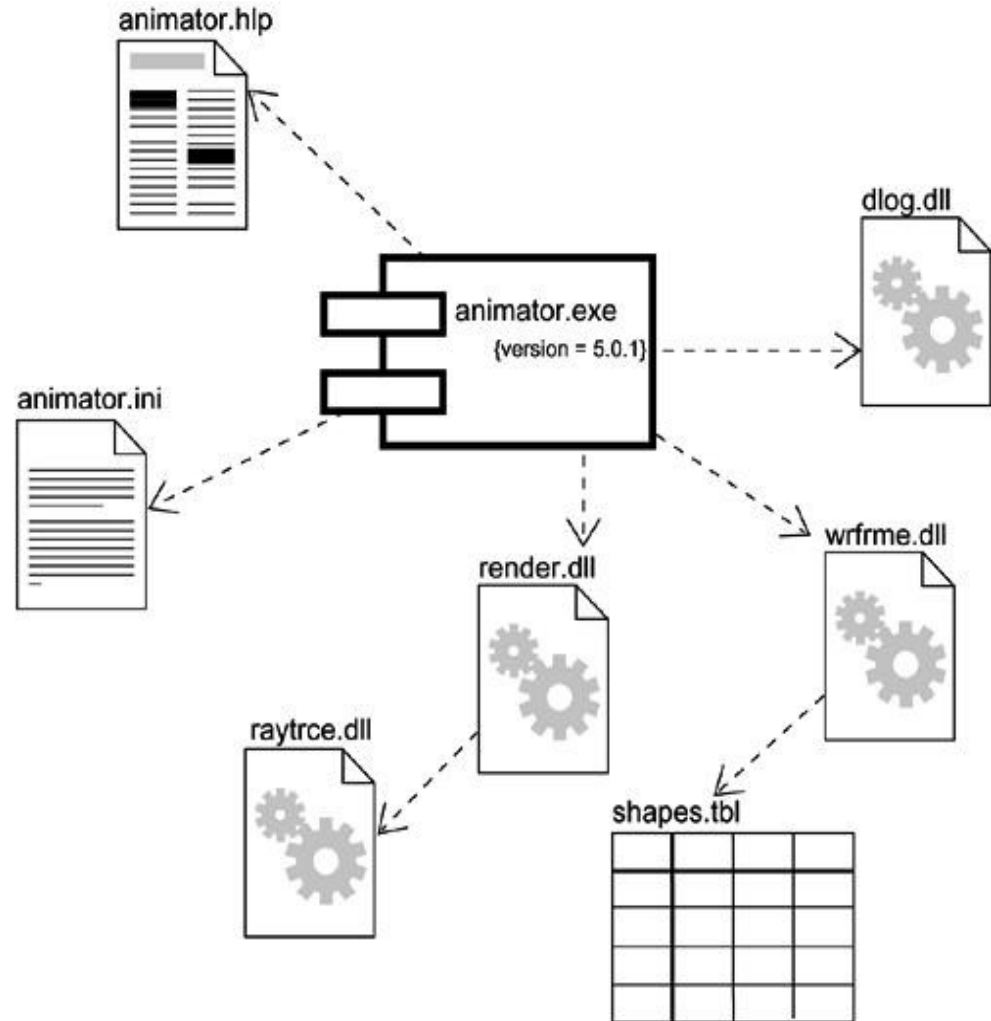
2. Object Diagram

- An *object diagram* shows a **set of objects and their relationships**.
- Object diagrams address the static design view or static process view of a system just as do class diagrams.



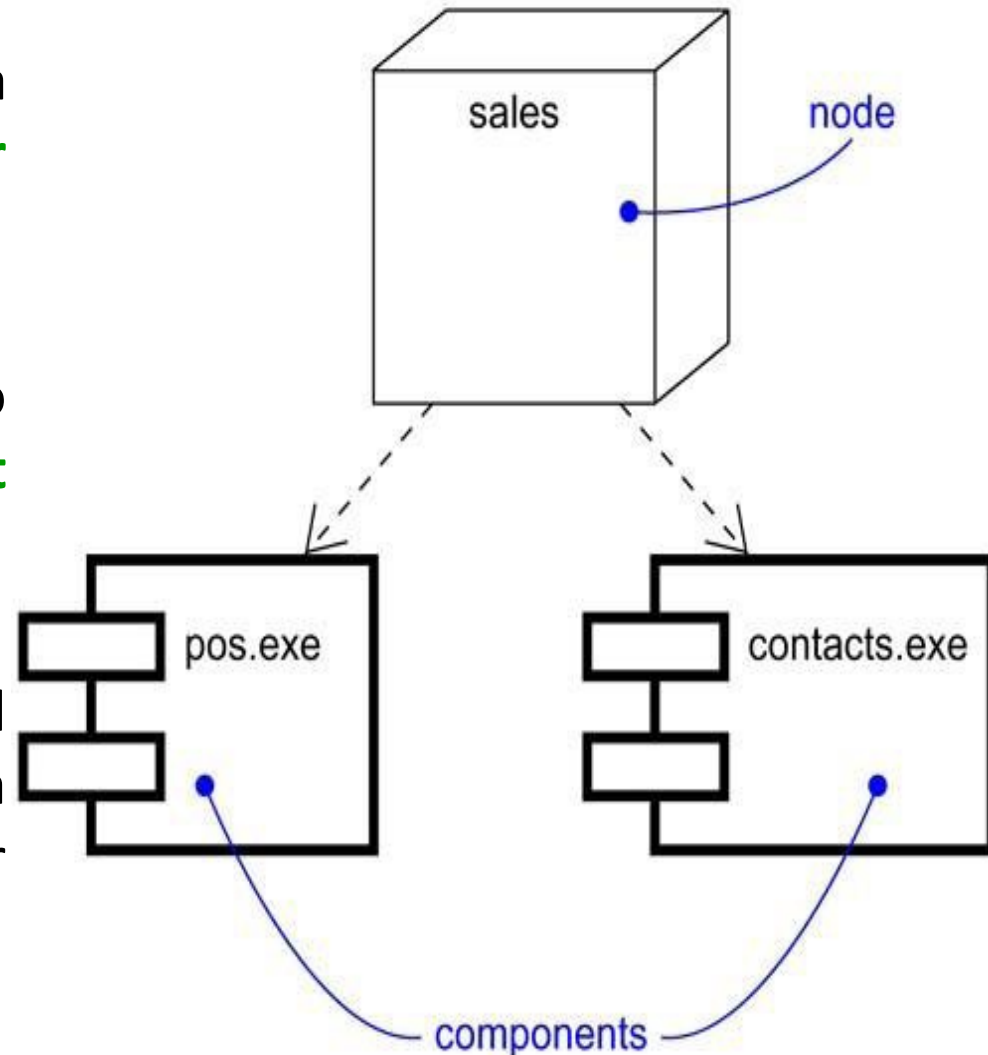
3. Component Diagram

- A **component diagram** shows a **set of components and their relationships**.
- Component diagrams are used to illustrate the **static implementation view** of a system.
- Component diagrams are related to class diagrams in that a component typically maps to one or more classes, interfaces, or collaborations.



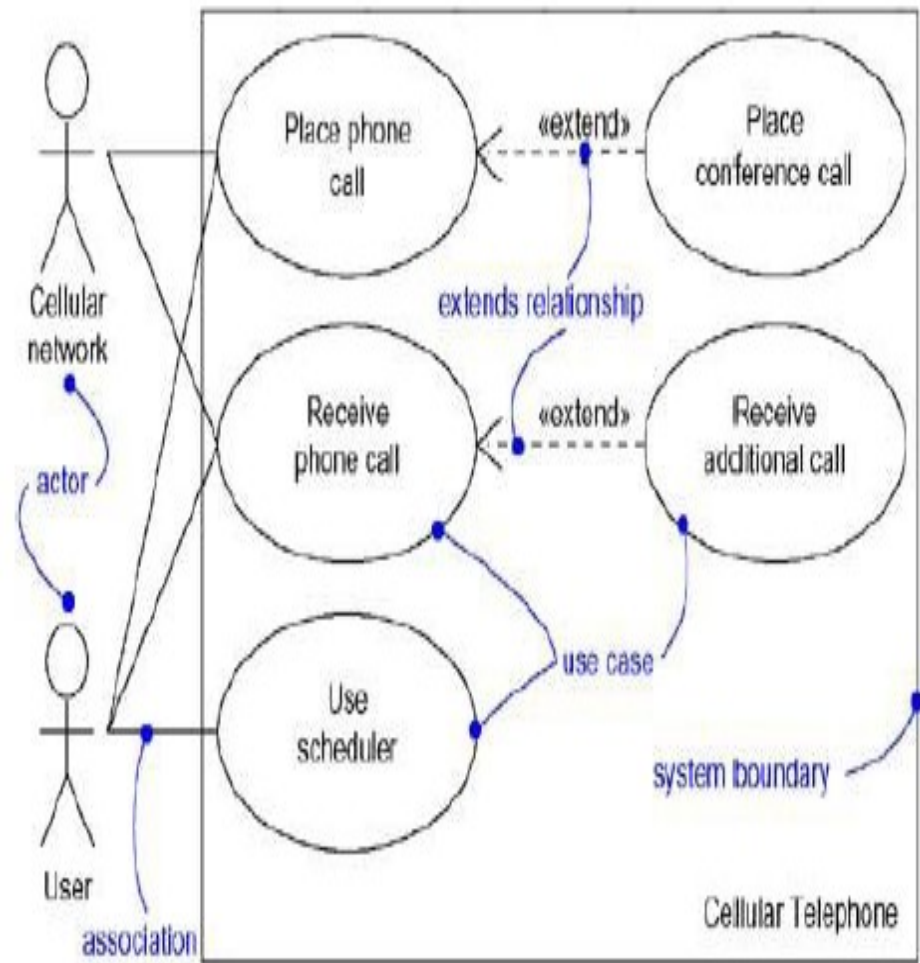
4. Deployment Diagram

- A **deployment diagram** shows a **set of nodes and their relationships**.
- Deployment diagrams are used to illustrate the **static deployment view** of an architecture.
- Deployment diagrams are related to component diagrams in that a node typically encloses one or more components.



Use Case Diagram

- A *use case diagram* shows a set of **use cases and actors (a special kind of class) and their relationships**.
- Apply use case diagrams to illustrate the static use case view of a system.
- Use case diagrams are especially **important in organizing and modeling the behaviors** of a system.



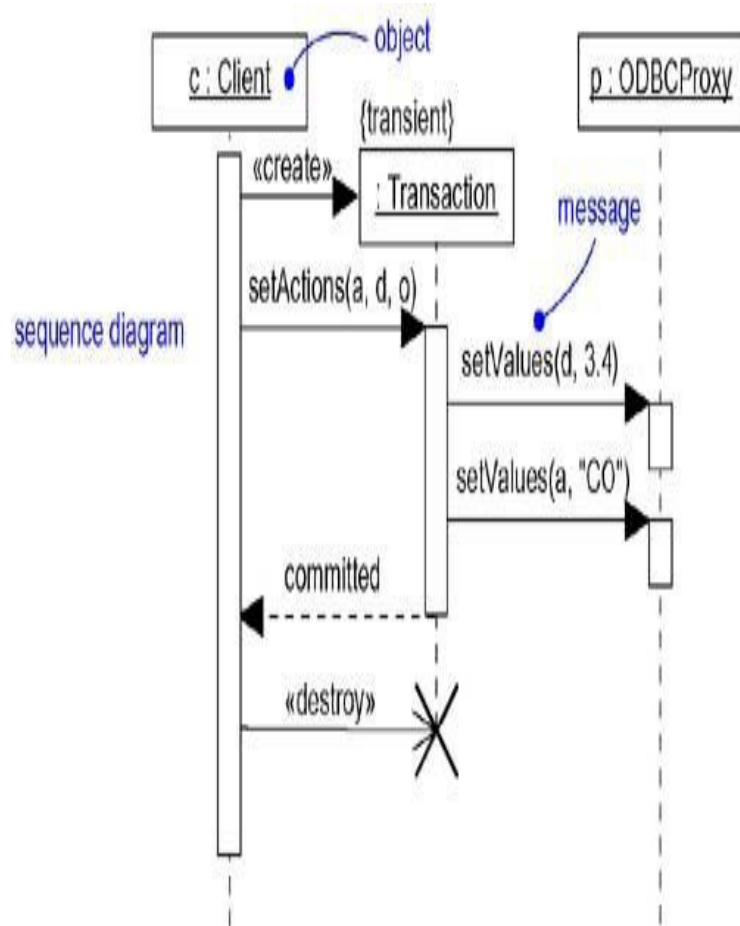
Interaction diagram

- *Interaction diagram* is the collective name given to sequence diagrams and collaboration diagrams.
- Both sequence diagrams and collaboration diagrams are kinds of interaction diagrams.
- An interaction diagrams shows an interaction, consisting of a set of objects and their relationships, including the messages that may be dispatched among them.
- Interaction diagrams address the dynamic view of a system.

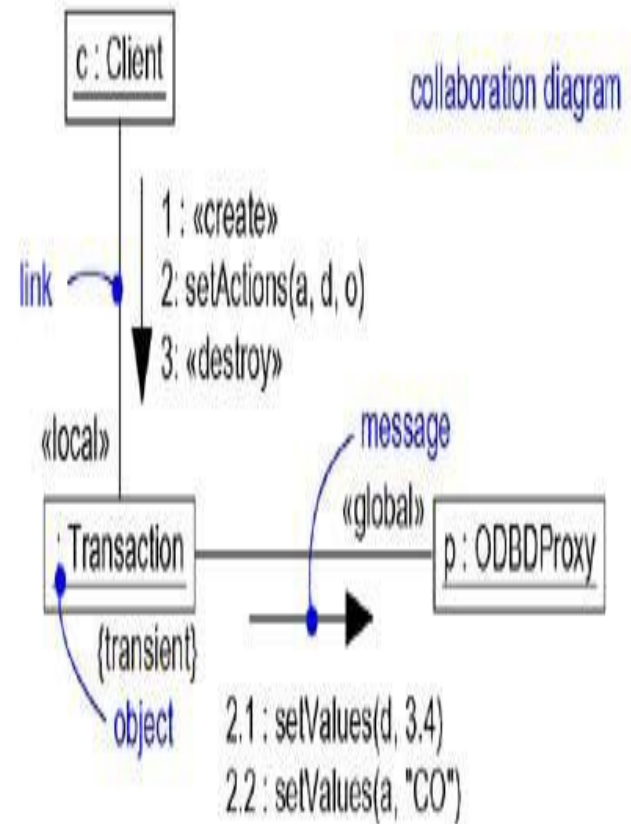
Interaction Diagrams

- A **sequence diagram** is an interaction diagram that emphasizes the **time ordering of messages**.
- A **collaboration diagram** is an interaction diagram that emphasizes the **structural organization** of the objects that send and receive messages.
- **Sequence diagrams and collaboration diagrams are isomorphic**, meaning that **you take one and transform it into the other**.

1. Sequence Diagram

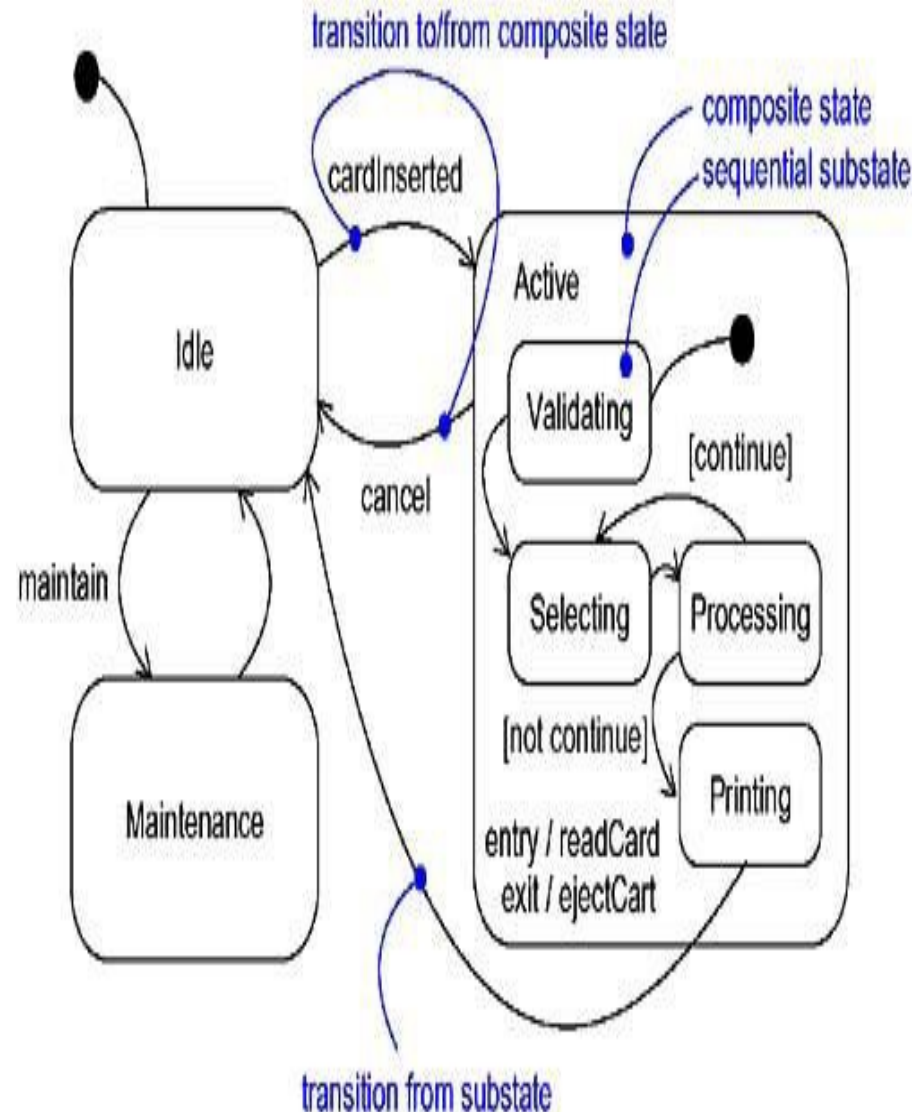


2. Collaboration Diagram



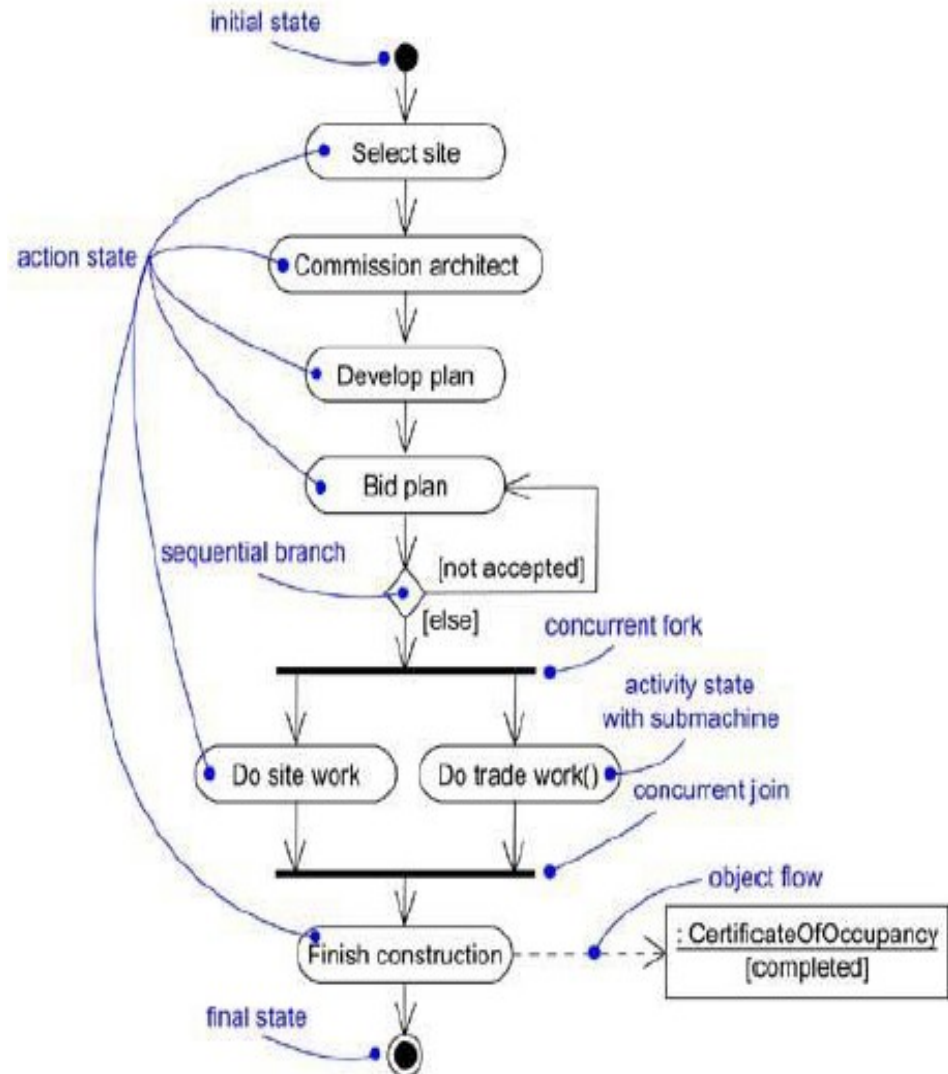
Statechart Diagram

- A **statechart diagram** shows a **state machine, consisting of states, transitions, events, and activities.**
- Use statechart diagrams to illustrate the **dynamic view of a system.**
- Statechart diagrams emphasize the **event-ordered behavior of an object**, which is especially useful in modeling reactive systems.



Activity Diagram

- An **activity diagram** shows the **flow from activity to activity within a system**.
- An activity shows a **set of activities, the sequential or branching flow from activity to activity**.
- Use activity diagrams to illustrate the **dynamic view** of a system.
- Activity diagrams emphasize the **flow of control among objects**.



Rules of the UML

- The UML's building blocks can't simply be thrown together in a random fashion.
- Like any language, the UML has a number of rules that specify what a well-formed model should look like.
- A *well-formed model* is one that is semantically self-consistent and in harmony (accordingly) with all its related models.

The UML has semantic rules for:-

- Names- What you can call things, relationships and diagrams.
- Scope- The context that gives specific meaning to a name.
- Visibility- How those names can be seen and used by others.
- Integrity- How things properly and consistently relate to one another.
- Execution- What it means to run or simulate a dynamic model.

- Models built during the development of a software-intensive system tend to evolve and may be viewed by many stakeholders in different ways and at different times.
- For this reason, it is common for the development team to not only build models that are well-formed, but also to build models that are

- **Elided** Certain elements are hidden to simplify the view
- **Incomplete** Certain elements may be missing
- **Inconsistent** The integrity of the model is not guaranteed

3.Common Mechanisms

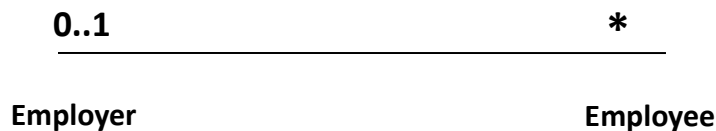
- The 4 common mechanisms that apply consistently throughout the language.
 1. Specifications
 2. Adornments
 3. Common Divisions
 4. Extensibility mechanisms

1. Specifications:-

- The UML is more than just a graphical language. Rather, behind every part of graphical notation there is a specification that provides a textual statement of the syntax and semantics of that building block.
- For example, behind a class icon is a specification that provides the full set of attributes, operations and behaviors.
- You use the UML's graphical notation to visualize a system; you use the UML's specification to state the system details.

2. Adornments:-

- Adornments **Textual/graphical items** added to the basic notation of an element.
- Used for visualizing details from the element's specification
- Example: The basic notation of association is a line, but this could be adorned with additional details, such as the role and multiplicity of each end.
- The most important kind of adornments are notes.



■

<i>Transaction</i>
+ execute() + rollback() # priority() - timestamp()

3.Common Divisions

➤ Abstraction vs. manifestation

- Class vs. object
- Most UML building blocks have this kind of class/object distinction.

➤ Interface vs. implementation

- An interface declares a contract, and an implementation represents one concrete realization of that contract.

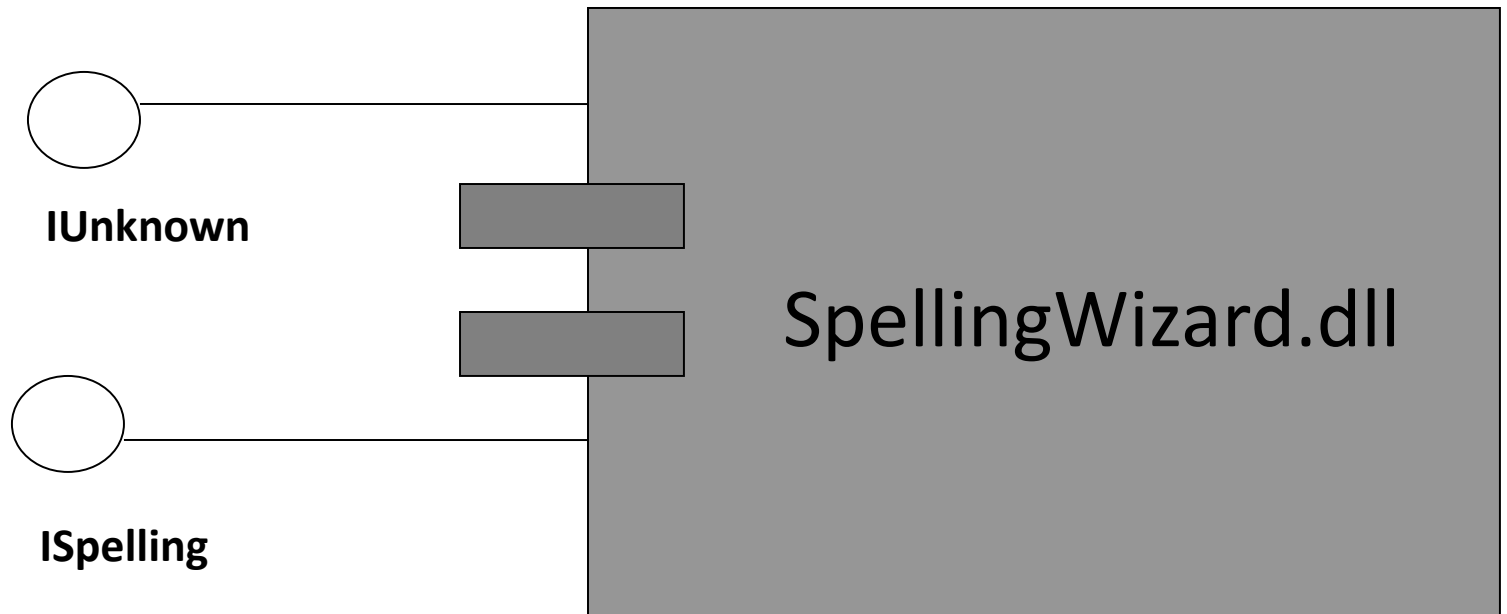
Customer
name
address
phone

Jan :Customer

:Customer

Elyse

Classes and Objects



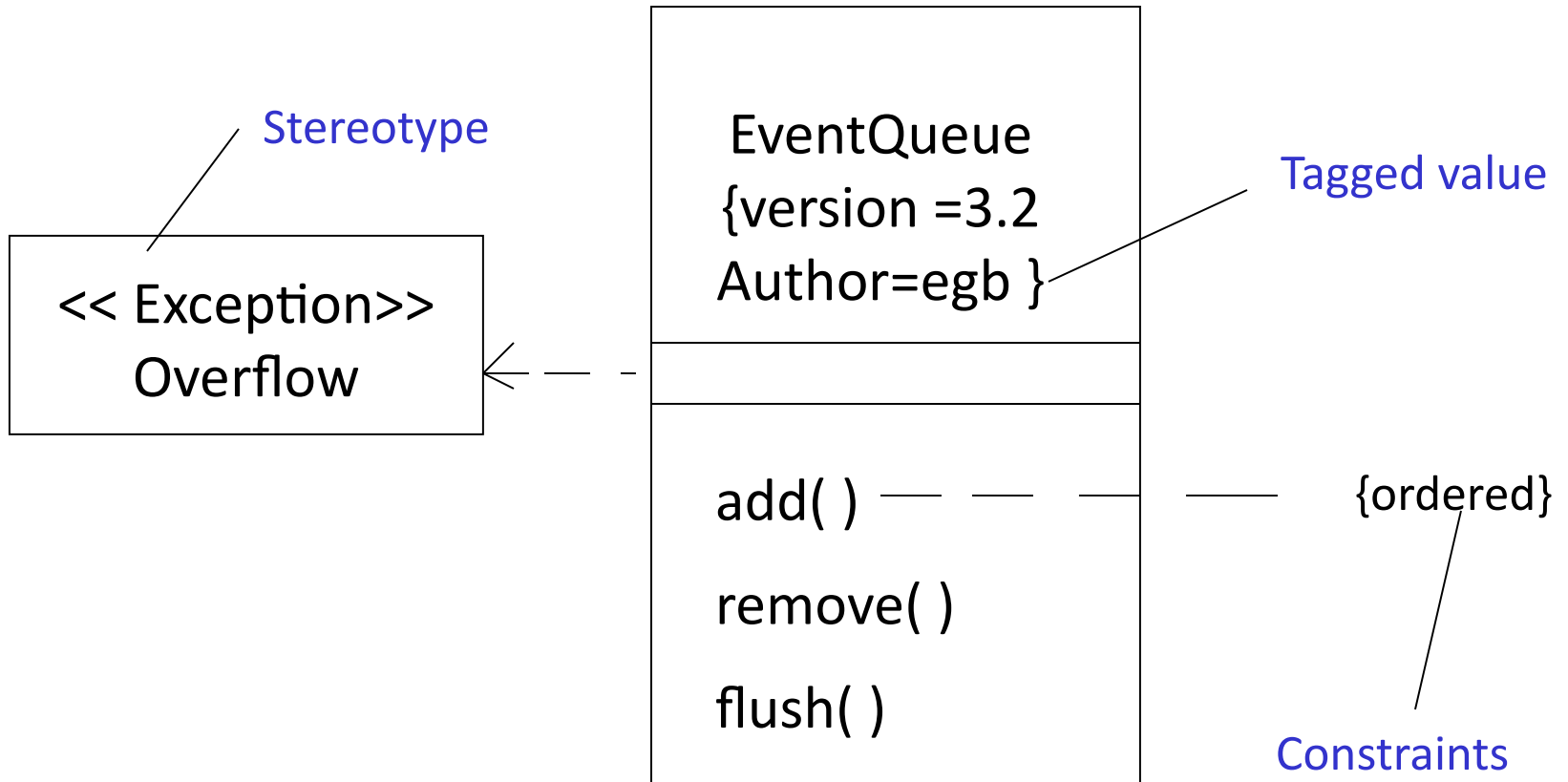
Interfaces and Implementations

4.Extensibility Mechanisms

- Allows you to extend the language by adding new building blocks, creating new properties and specifying new semantics. Includes:
 1. Stereotypes
 2. Tagged values
 3. Constraints

- Stereotypes

- Extend the **vocabulary of the UML** by creating new model elements derived from existing ones but that have **specific properties** suitable for your domain/problem.

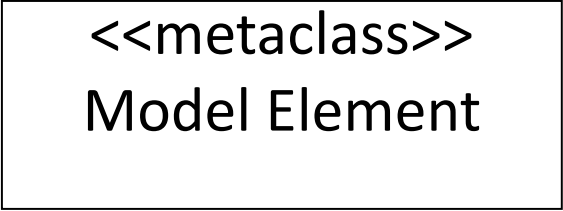


Extensibility Mechanisms

- Example: In Java, you sometimes have to model classes such as **exceptions**.
- Only want them to be thrown and caught
- Can make them first class citizens in your model, ie treating them like basic building blocks, by marking them with a suitable stereotype.

- Graphically, a stereotype is rendered as a name enclosed by guillemots and placed above the name of another element (eg, <<name>>)
- Alternatively, you can render the stereotyped element by using a new icon associated with that stereotype

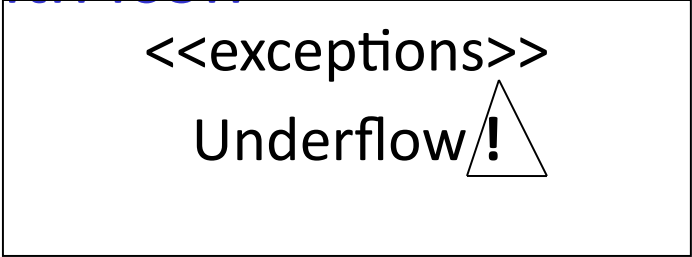
- Named stereotype



<<metaclass>>
Model Element

A rectangular box representing a metaclass. The text "<<metaclass>>" is on the top line, and "Model Element" is on the bottom line.

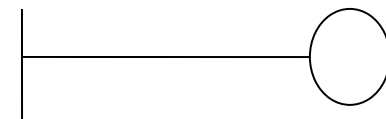
- Named stereotype with icon



<<exceptions>>
Underflow! 

A rectangular box representing an exception. The text "<<exceptions>>" is on the top line. The bottom line contains the text "Underflow!" followed by a warning triangle icon.

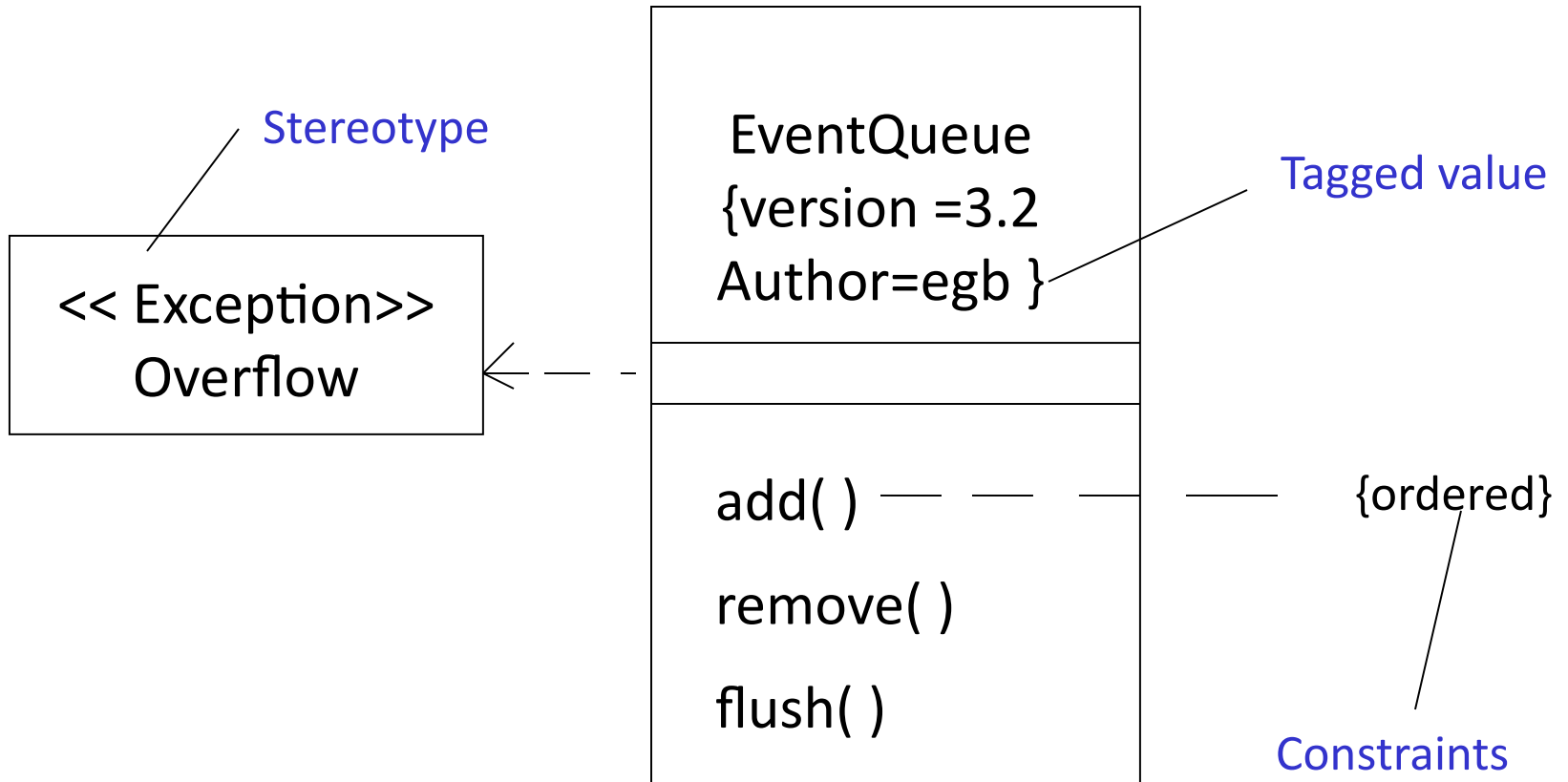
- Stereotyped element as icon



HumiditySensor

2.Tagged values:-

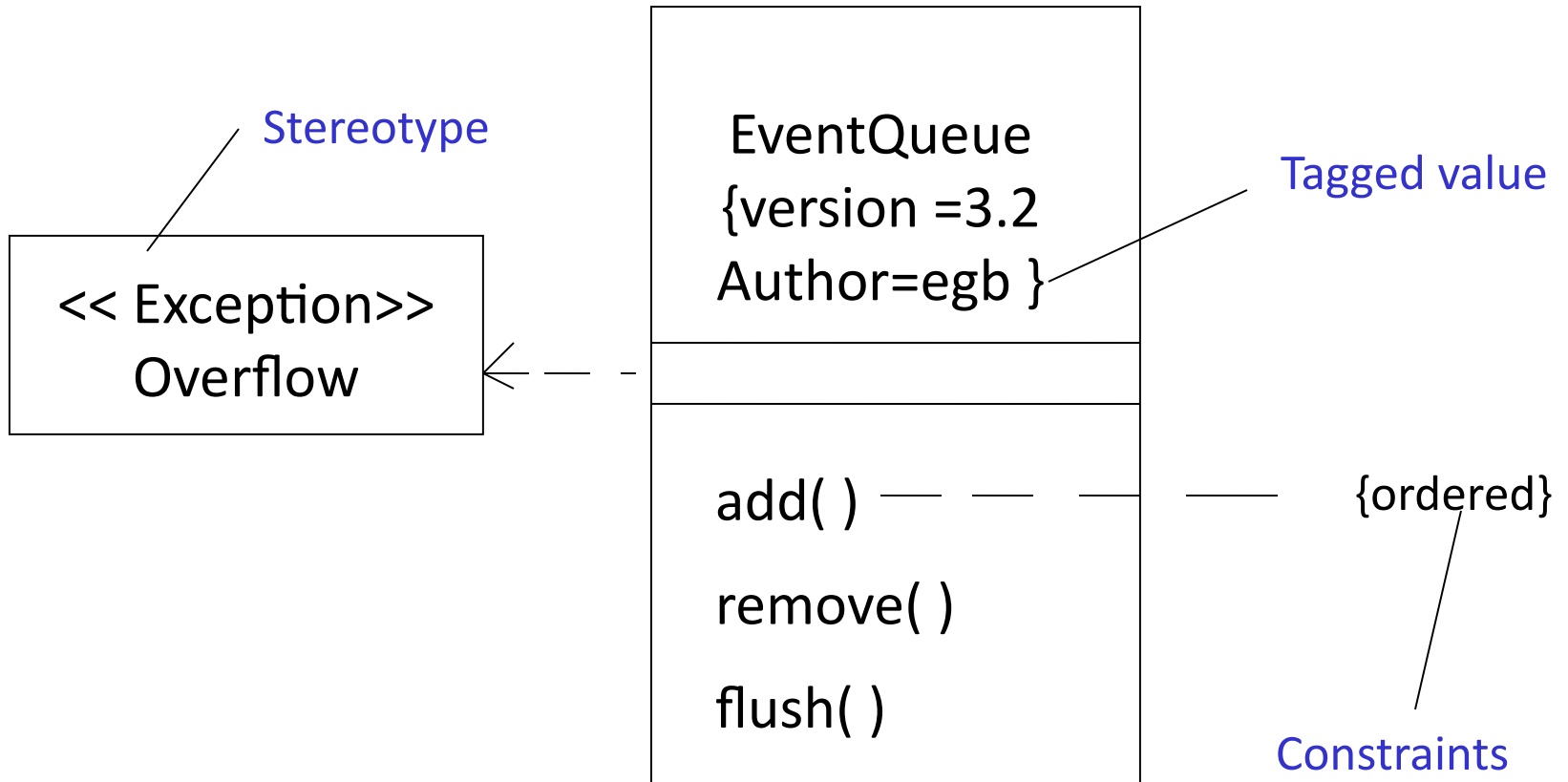
- Properties for specifying key-value pairs of model elements, where keywords are attributes.
- Extend the properties of a UML building block, allowing you to create new information in that elements specification.
- Can be defined for existing elements of the UML



Extensibility Mechanisms

3.Constraints:-

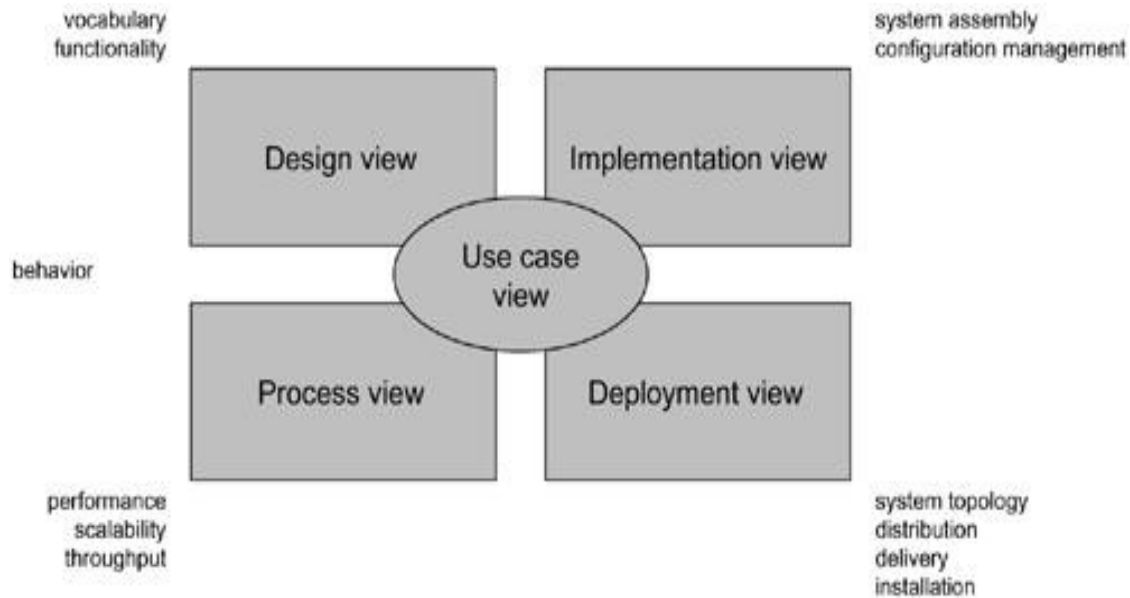
- Properties for specifying semantics or conditions that must be maintained as true for model elements.
- Extend the semantics of a UML building block, allowing you to add new rules, or modify existing ones.
- Example:- you might want to constrain the EventQueue class so that all additions are done in order.



Extensibility Mechanisms

Architecture

- Modeling a System's Architecture



Use case view

- The *use case view* of a system includes the **use cases** that describe the behavior of the system as seen by its end users, analysts, and testers.
- This view doesn't specify the organization of a software system. Rather, it Specify the **shape of the system's architecture**.
- The **static aspects** of this view are captured in **use case diagrams**; and the **dynamic aspects** of this view are captured in **interaction diagrams, statechart diagrams, and activity diagrams**.

Design view

- The ***design view*** of a system includes the classes, interfaces, and collaborations that form the vocabulary of the problem and its solution.
- This view primarily **supports the functional requirements** of the system, meaning the **services that the system should provide to its end users**.
- The **static aspects** of this view are captured in **class diagrams and object diagrams** and the **dynamic aspects** of this view are captured in **interaction diagrams, statechart diagrams, and activity diagrams**.

Process view

- The ***process view*** of a system includes the threads and processes that form the system's concurrency and synchronization mechanisms.
- This view primarily addresses the performance, scalability, and throughput of the system.
- The static and dynamic aspects of this view are captured in the same kinds of diagrams as for the design view, but with a focus on the active classes that represent these threads and processes.

Implementation view

- The *implementation view* of a system includes the components and files that are used to assemble and release the physical system.
- This view primarily addresses the **configuration management of the system's releases**, made up of somewhat independent components and files that can be assembled in various ways to produce a running system.
- The **static aspects** of this view are captured in component diagrams; the **dynamic aspects** of this view are captured in **interaction diagrams, statechart diagrams, and activity diagrams**.

Deployment view

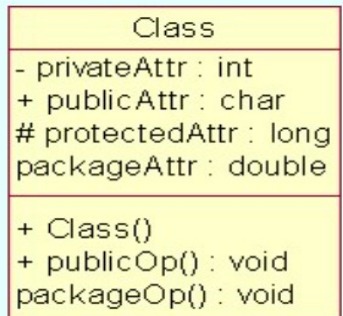
- The ***deployment view*** of a system includes the nodes that form the system's hardware topology on which the system executes.
- This view primarily addresses the **distribution, delivery, and installation of the parts that make up the physical system.**
- The **static aspects** of this view are captured in **deployment diagrams**; the **dynamic aspects** of this view are captured in **interaction diagrams, statechart diagrams, and activity diagrams.**

Difference between Forward Engineering and Reverse Engineering

Forward Engineering vs. Reverse Engineering

Forward Engineering	Reverse Engineering
• In forward engineering, the application are developed with the given requirements.	• In reverse engineering or backward engineering, the information are collected from the given application.
• Forward Engineering is a high proficiency skill.	• Reverse Engineering or backward engineering is a low proficiency skill.
• Forward Engineering takes more time to develop an application.	• While Reverse Engineering or backward engineering takes less time to develop an application.
• The nature of forward engineering is Prescriptive.	• The nature of reverse engineering or backward engineering is Adaptive.
• In forward engineering, production is started with given requirements.	• In reverse engineering, production is started by taking the existing products.

Class



- UML model -

```
public class Class
{
    private int privateAttr;
    public char publicAttr;
    protected long protectedAttr;
    double packageAttr;

    public Class() {}
    public void publicOp() {}
    void packageOp() {}
}
```

- Java code generated by FE -

Classes and associations

```
public class C
{
    private int privateAttr;
    public char publicAttr;
    C() {}
    public void publicOp() {}
    void packageOp() {}
}

class A {
    private long privateAttr;
    public C c;
    A() {}
}

class B {
    protected int protectedAttr;
    public C[] cs;
    B() {}
}
```

- Java source code -

```
classDiagram
    class C {
        -privateAttr : int
        +publicAttr : char
        C()
        +publicOp() : void
        packageOp() : void
    }
    class A {
        -privateAttr : long
        A()
    }
    class B {
        #protectedAttr : int
        B()
    }
    A --> C : +c
    B --> C : +cs[]
```

Each attribute will be represented as an association if its class is represented in the model.

- UML model generated by RE -

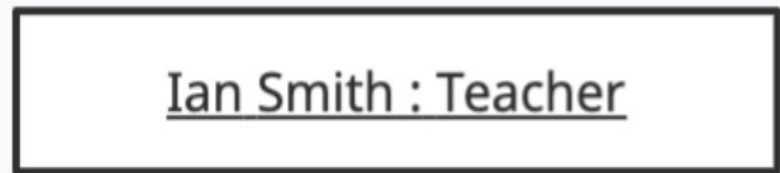
SEQUENCE DIAGRAM

- A **Sequence Diagram** is a type of diagram defined in the **Unified Modeling Language (UML)** that shows how objects or components interact with each other over time.
- It focuses on the **time sequence of messages** exchanged between objects to perform a function or process.

- **Key Features of a Sequence Diagram:**

- **Actors/Objects**

- Represented at the top as rectangles with names (e.g., *User*, *System*, *Database*).
- Placed horizontally.



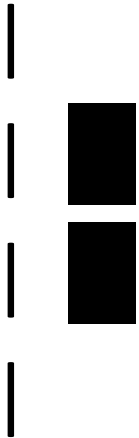
- **Lifelines**

- Vertical dashed lines extending downward from each object/actor.
- Show the passage of time.

User System



- **Activation bars**
- Thin rectangles on a lifeline.
- Indicate when an object is active or performing a task.



Sequence diagram Messages

- 1. Synchronous messages**
- 2. Asynchronous Messages**
- 3. Create message**
- 4. Delete Message**
- 5. Self Message**
- 6. Reply Message**

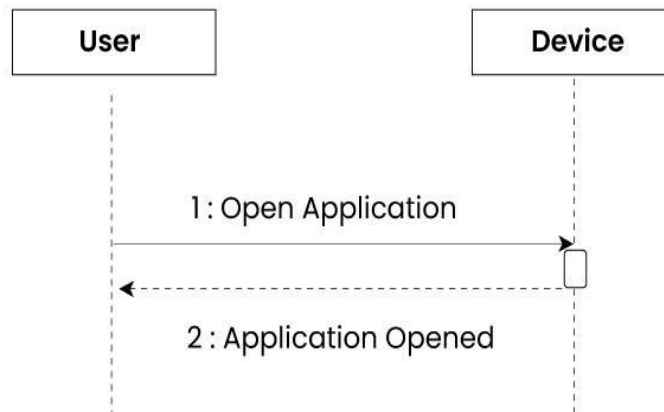
1. Synchronous messages

A synchronous message waits for a reply before the interaction can move forward. The sender waits until the receiver has completed the processing of the message. The caller continues only when it knows that the receiver has processed the previous message

i.e. it receives a reply message.

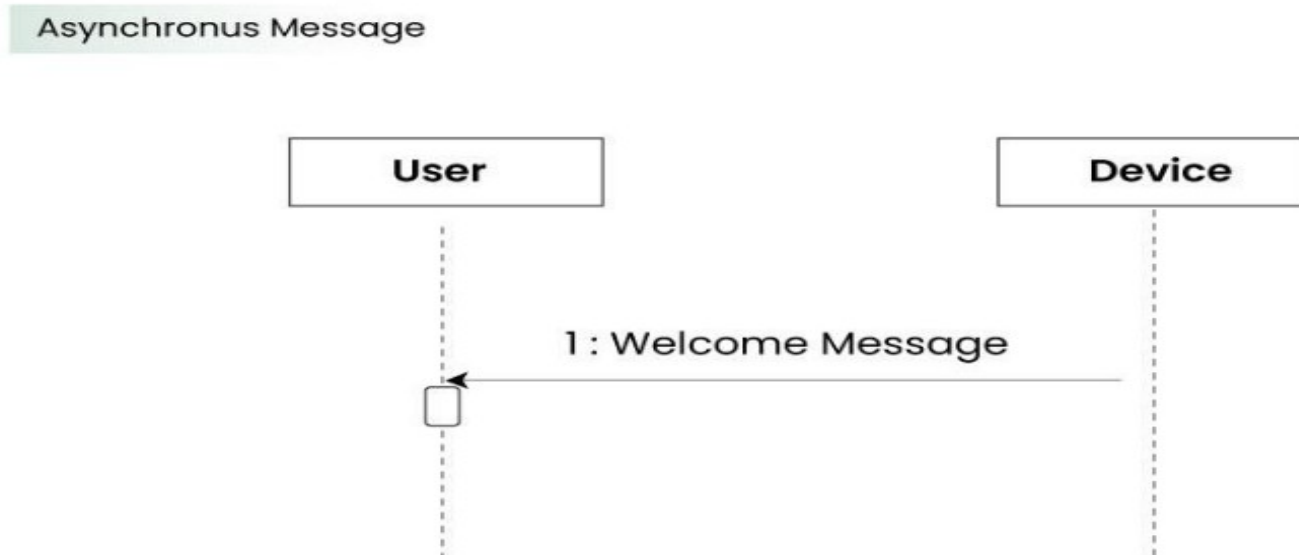
- A large number of calls in object oriented programming are synchronous.
- We use a **solid arrow head** to represent a synchronous message.

Synchronus Message



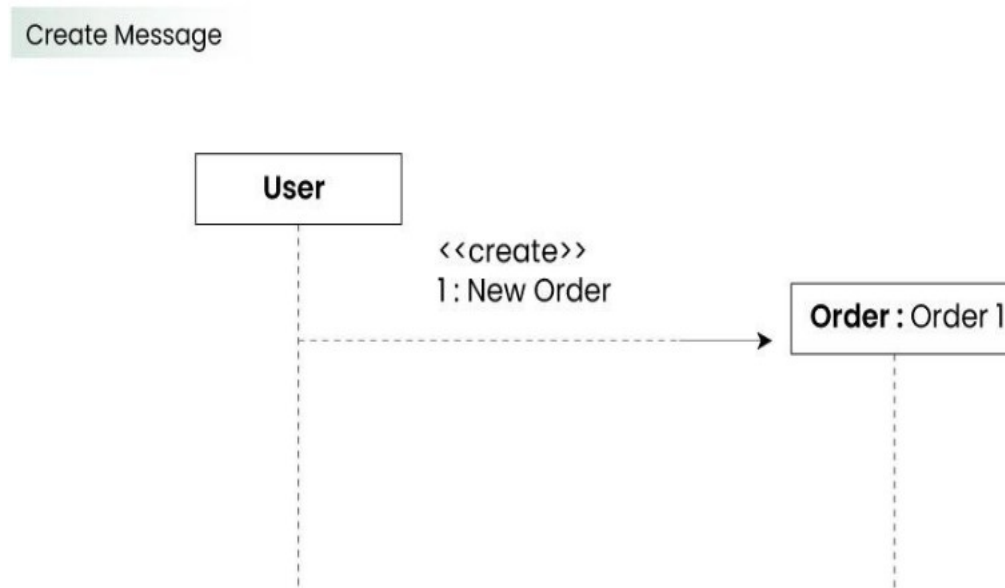
- **2. Asynchronous Messages**

An asynchronous message does not wait for a reply from the receiver. The interaction moves forward irrespective of the receiver processing the previous message or not. We use a lined arrow head to represent an asynchronous message.



3. Create message

We use a Create message to instantiate a new object in the sequence diagram. There are situations when a particular message call requires the creation of an object. It is represented with a dotted arrow and create word labelled on it to specify that it is the create Message symbol.

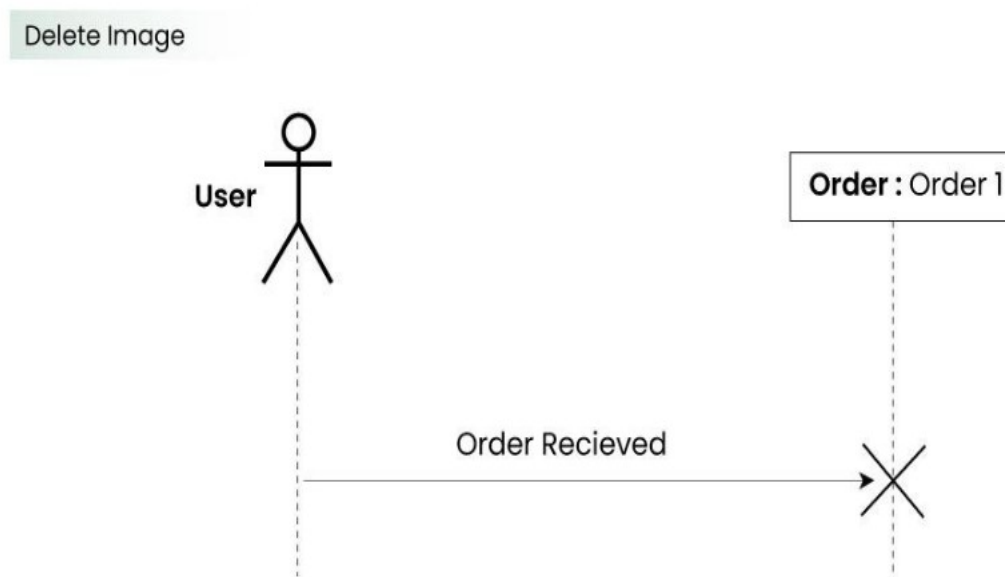


4. Delete Message

We use a Delete Message to delete an object. When an object is deallocated memory or is destroyed within the system we use the Delete Message symbol. It destroys the occurrence of the object in the system. It is represented by an arrow terminating with a x.

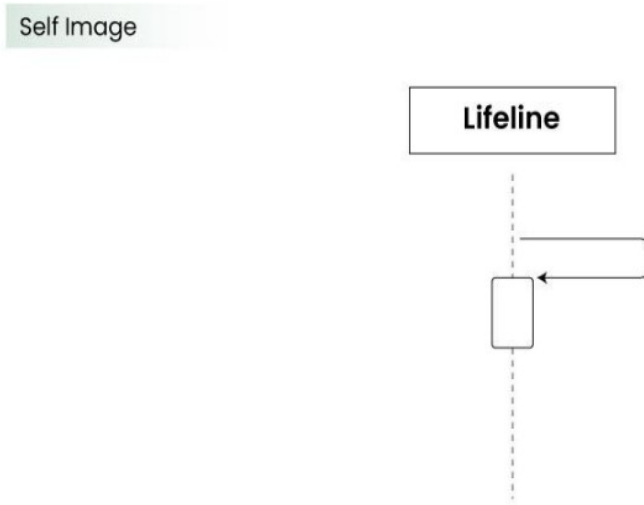
For example:

In the scenario below when the order is received by the user, the object of order class can be destroyed.



5. Self Message

Certain scenarios might arise where the object needs to send a message to itself. Such messages are called Self Messages and are represented with a **U shaped arrow**.



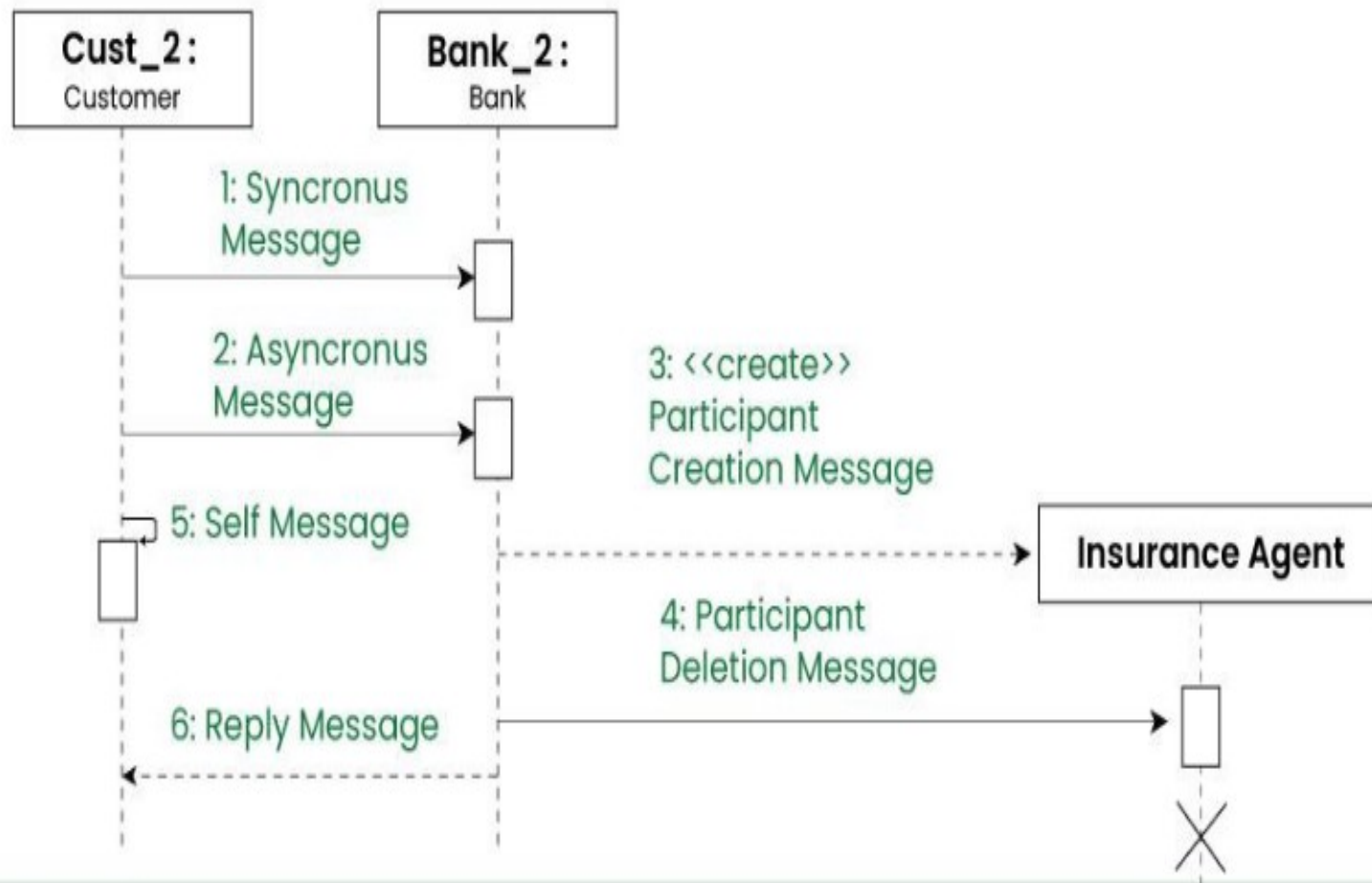
6. Reply/Return Message

Reply messages are used to show the message being sent from the receiver to the sender. We represent a return/reply message using an **open arrow head with a dotted line**. The interaction moves forward only when a reply message is sent by the receiver.

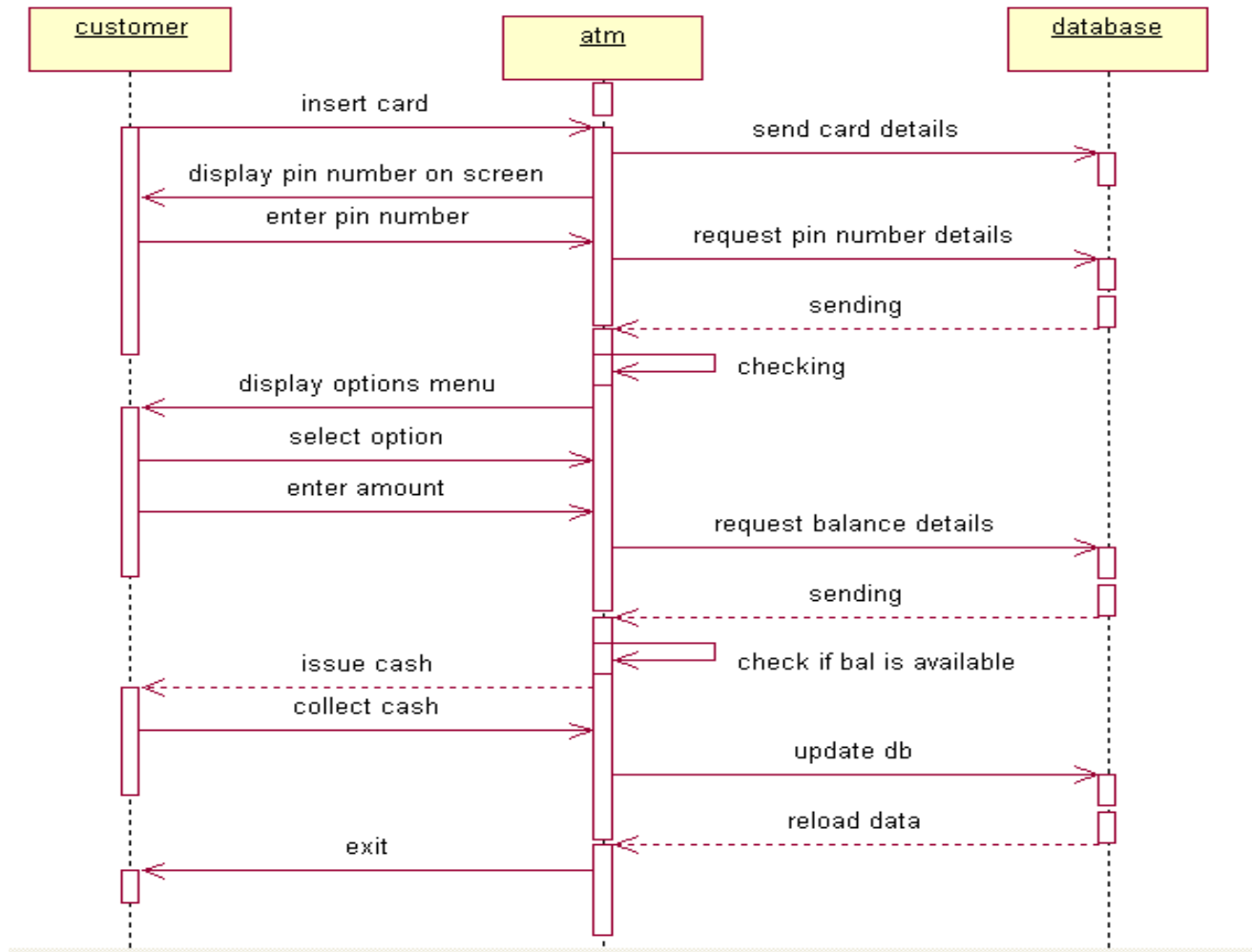
Reply Message



Different Types of Messages



Example for sequence diagram



END